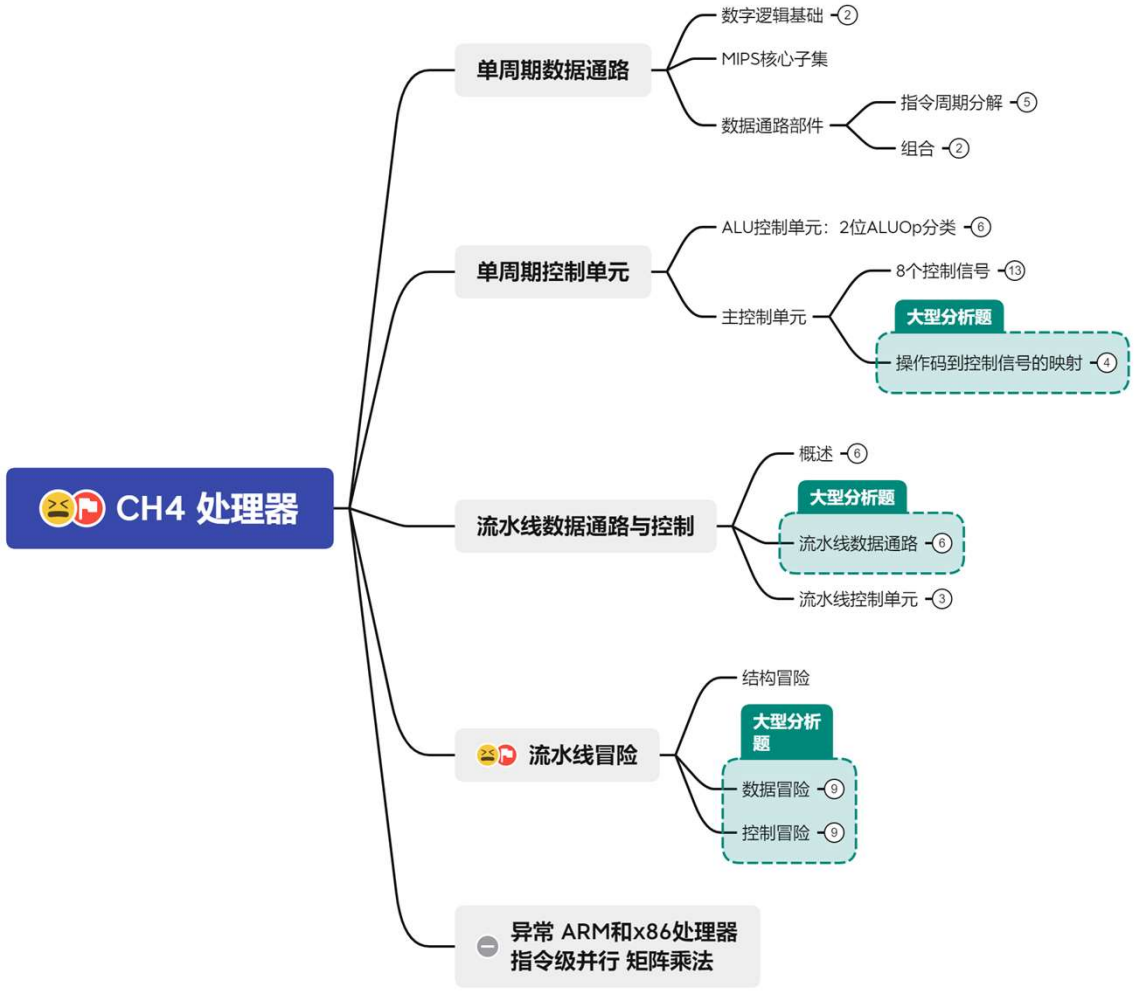
CH4 处理器

课程基于
《计算机组成与设计：硬件/软件接口》5e
Patterson & Hennesy 著

B站 翼云图灵

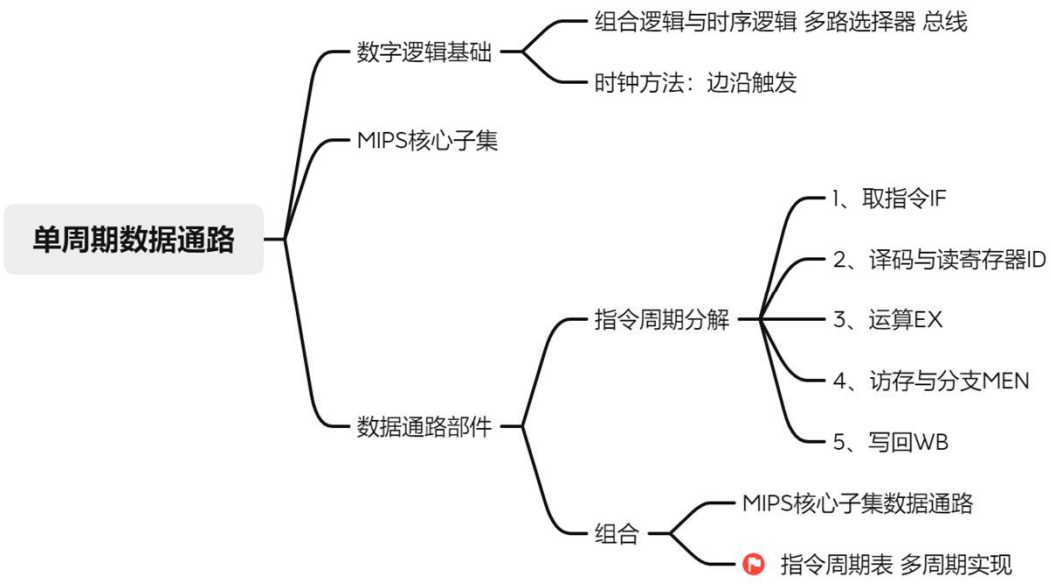
章节导图


CH4 处理器
单周期数据通路
数字逻辑基础 2
MIPS核心子集
数据通路部件
指令周期分解 5
组合 2
单周期控制单元
ALU控制单元：2位ALUOp分类 6
主控制单元
8个控制信号 13
大型分析题
操作码到控制信号的映射 4
流水线数据通路与控制
概述 6
大型分析题
流水线数据通路 6
流水线控制单元 3
流水线冒险
结构冒险
大型分析题
数据冒险 9
控制冒险 9
异常 ARM和x86处理器
指令级并行 矩阵乘法

翼云图灵

第一部分

单周期数据通路



B站 翼云图灵

数字逻辑基础

根据是否含有存储器，数字逻辑部件分为两类：

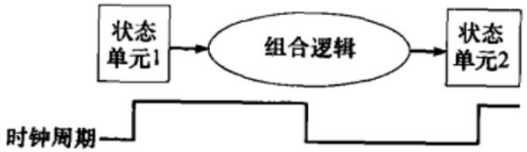
组合逻辑不含存储器，对数据进行操作，给定输入时输出唯一确定

状态逻辑（又称时序逻辑）含有存储器

至少拥有两个输入：时钟、待写入的数据——和一个输出：存储着的数据

只有在时钟信号的上升沿，才允许向状态单元写入数据

这样的时钟方法称为边沿触发的时钟（edge-triggered clock）



多选器（MUX）从多个数据中选择一个作为输出

选择哪个数据取决于选择控制信号

总线（bus）表示数据信息多于一位的信号线，在数字电路图中加粗表示并标记位宽

因为MIPS-32采用32位字，当总线为32位时不必写出位宽

B站 翼云图灵

MIPS核心子集 指令周期

选取以下9条指令构成一个MIPS核心子集，作为本章实现的指令集：

R型指令：add、sub、AND、OR、slt

访存指令：lw、sw

决策指令：beq、j（后续单独扩充实现）

使用这个MIPS子集可以说明建立数据通路和控制单元的关键原理

和其他体系结构的基本思想是相通的

一条MIPS指令的执行分为五个阶段，统称为一个指令周期（instruction cycle）

①IF取指令：根据PC所给地址，从存储器中取出指令

②ID译码与读寄存器：分析指令字段，读取一个或两个寄存器

③EX运算：ALU运算R型指令的结果/访存指令的地址/beq两源操作数是否相等

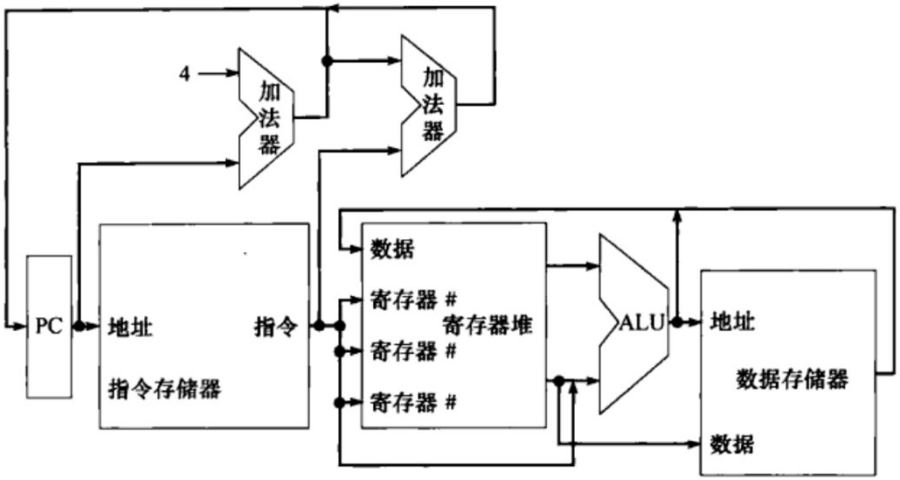
④MEM访存与分支：访存指令向存储器进行读写，分支指令完成分支

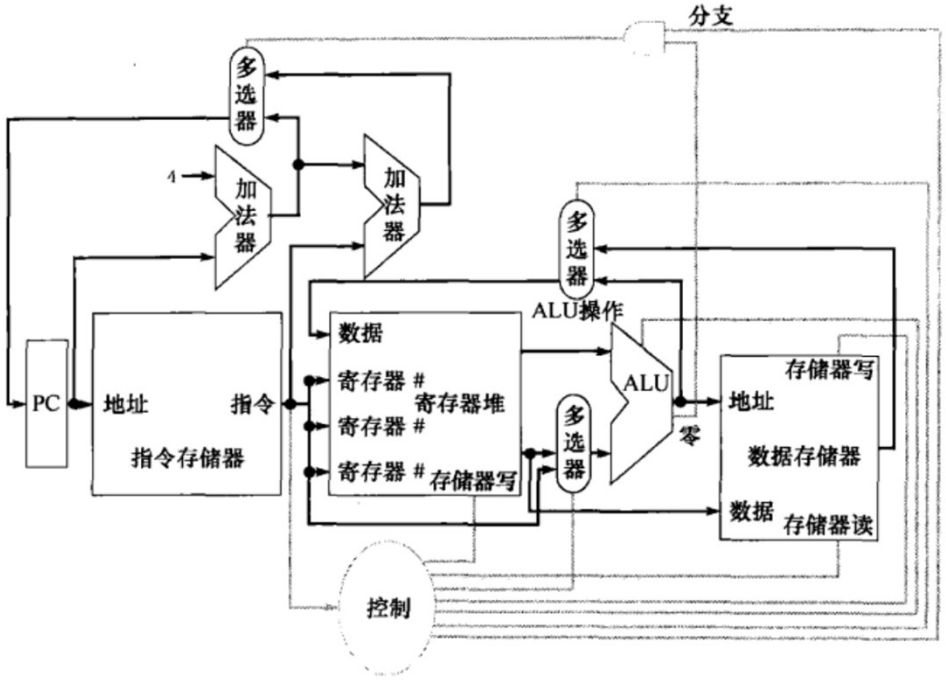
⑤WB写回：将结果送回某寄存器

B站 翼云图灵

数据通路概图

左图表示数据的流向，右图在左图的基础上添加了必要的多选器和控制信号





B站 翼云图灵

数据通路部件：取指令周期IF

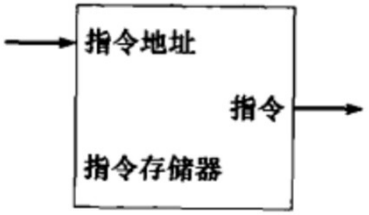
①IF取指令：根据PC所给地址，从存储器中取出指令

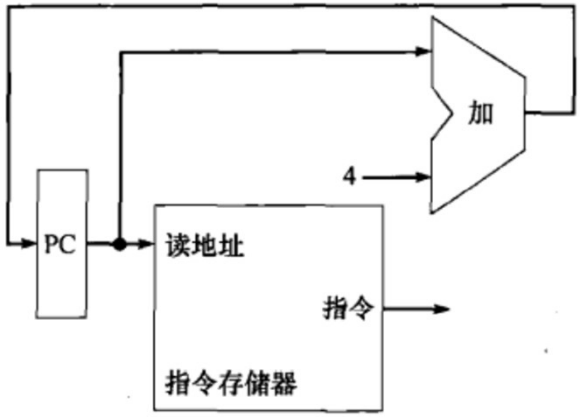
我们将存储器中的指令和数据分别看待，由此形成指令存储器和数据存储器

指令存储器存放程序的指令，输入一个地址时输出其指向的指令

程序计数器（PC）保存当前指令的地址

我们将一个ALU完全用作加法器（Add），用来计算PC+4，即下一条指令的地址



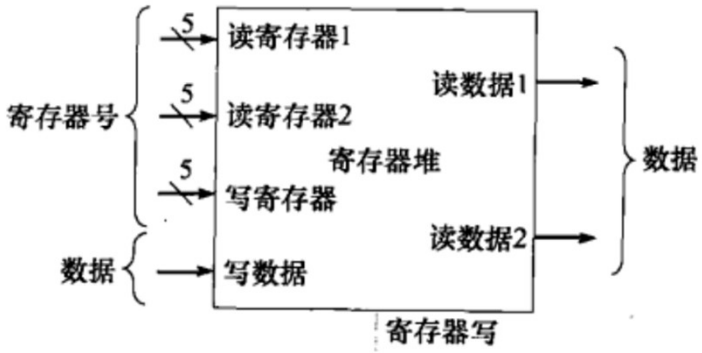


B站 翼云图灵

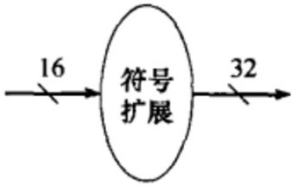
数据通路部件：译码与读寄存器周期ID

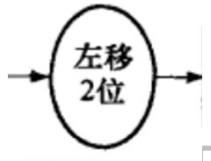
②ID译码与读寄存器：分析指令字段，读取一个或两个寄存器

寄存器堆（reg file）含有32个通用寄存器
对应R型指令的三操作数格式，寄存器堆接收3个寄存器编号
其中至多两个寄存器用于读（R型和beq读两个，sw读一个）
至多一个寄存器用于写（R型写回rd、lw写回rt）
读取时输出rs和rt寄存器中的数据
为了支持写入数据，需要一个32位的写数据端口作为输入



访存指令中的16位偏移量（立即数）和beq中的相对地址
需要经符号扩展（sign-extend）逻辑扩充到32位
对访存指令，这个数和rs中的基地址相加
对beq指令，需要一个移位逻辑将字地址转换为字节地址





B站 翼云图灵

数据通路部件：运算周期EX

③EX运算：ALU运算R型指令的结果/访存指令的地址/beq两源操作数是否相等

作出IF、ID、EX阶段的局部数据通路

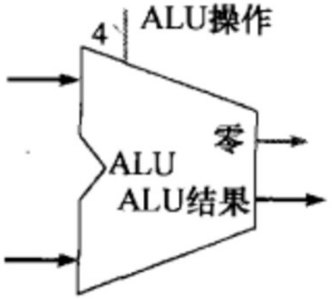
①对于R型指令，ALU执行相应的算数/逻辑运算，并输出结果

②对于访存指令，ALU计算基地址和偏移量的和，得到数据的真正地址

③对于分支指令，ALU将两源操作数相减，根据结果是否为0，判断两数是否相等

与此同时，加法器得到符号扩展并左移两位的PC相对地址

将其与PC+4相加得到分支目标地址



B站 翼云图灵

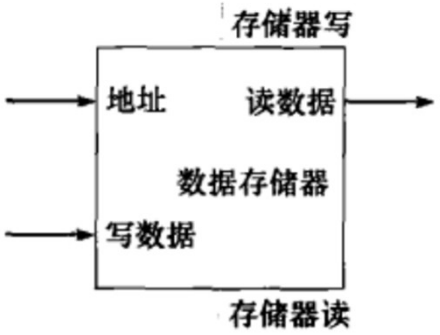
数据通路部件：访存与分支周期MEM、写回周期WB

④MEM访存分支：访存指令向存储器进行读写，分支指令完成分支
⑤WB写回：将结果送回某寄存器

访存指令在第四阶段才真正读/写数据存储器
ALU计算基址和偏移量的和，得到真正地址并输入数据存储器
lw从数据存储器读取数据并输出，sw向数据存储器写入数据

beq分支指令在这个周期不访存4
但会将PC+4/分支目标地址写回PC，决定分支是否发生
因此，将第四阶段统称为访存分支周期

第五阶段，R型指令将运算结果写回rd寄存器
lw指令将存储器数据写回rt寄存器



B站 翼云图灵

MIPS核心子集数据通路

B站 翼云图灵

该数据通路对教材图4-11作了一处补充：用多选器确定写回的寄存器是rt还是rd

指令周期小结

	IF取指令	ID译码与读reg	EX运算	MEM访存分支	WB写回
R型	根据PC 从存储器中 取出指令	取源操作数 rs和rt	执行运算	-	将ALU运算结果 写回rd
lw		取基址寄存器rs 偏移量符号扩展	计算内存地址	读数据存储器 取得数据	将访存数据 写回rt
sw				写数据存储器	-
beq		取rs和rt 字地址符号扩展 并左移两位	比较rs和rt 算分支目标地址	地址写回PC	-

在单周期实现中，每条指令都在一个时钟周期内完成，CPI为1

注意到只有lw使用全部5个阶段，因此基本可以肯定，时钟周期取决于lw的执行时长

其他指令只用4个阶段，但仍然要花费5个阶段的时间

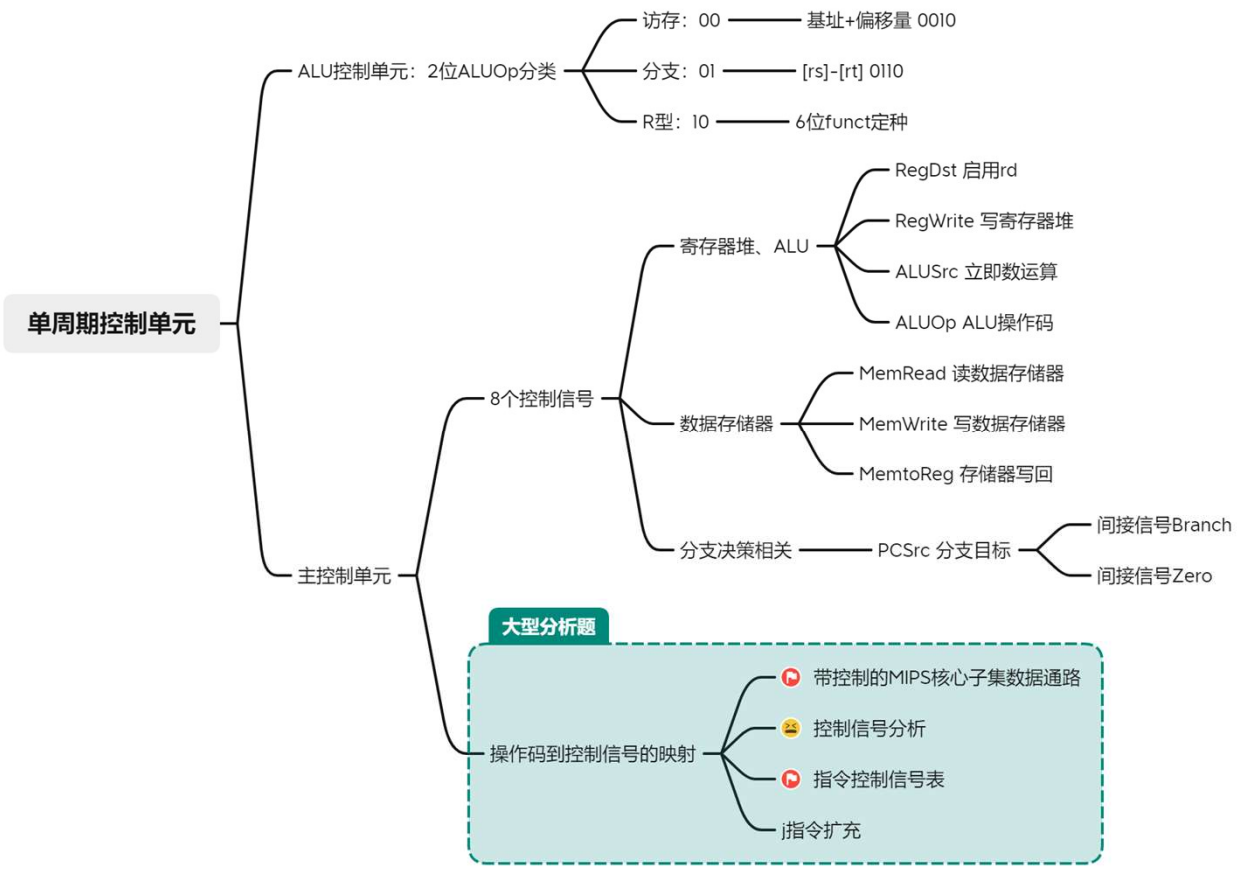
多周期实现可以缩减时钟周期到1个阶段的长度，一条指令占用几个阶段，就执行几个周期

虽然CPI变成了4或5，但时钟周期缩短到1/5，运行速度反而更快

B站 翼云图灵

第二部分

单周期控制单元



ALU控制线

ALU可以执行加add、减sub、与AND、或OR、小于则置位slt五种运算（或非NOR不讨论）

具体执行什么运算，由4位ALU控制线决定

Ainvert(1b) Bnegate(1b) Operation(2b)

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

当Ainvert、Bnegate都无效时

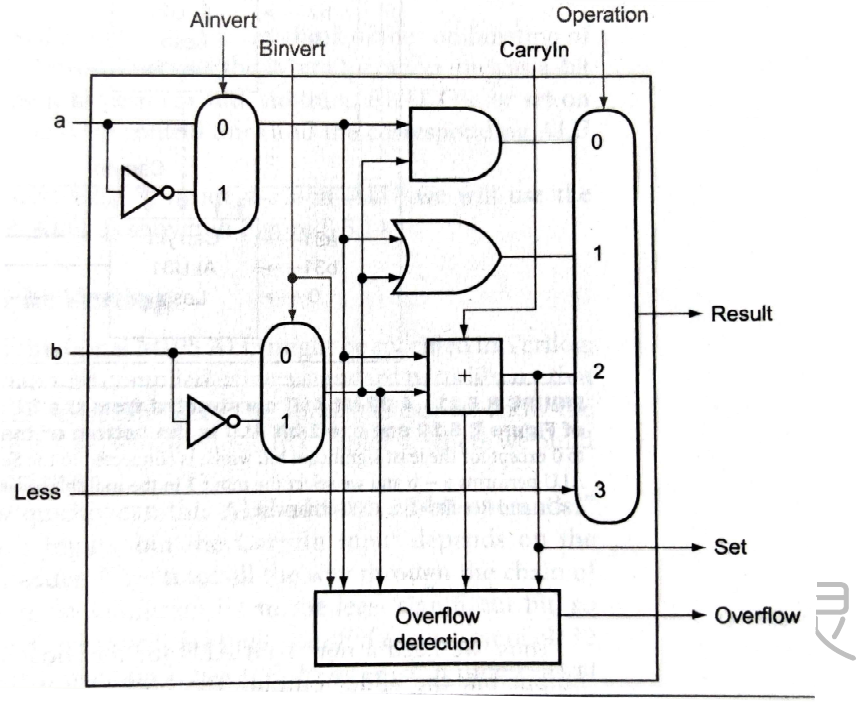
Operation控制右方多路选择器从

a&b，a|b，a+b中选择一个作为输出

当执行减法时，转化为补码加法

由Binvert和CarryIn合二为一的Bnegate有效

a-b = a+(-b) = a+(binvert+1)



ALU控制单元

主控制单元根据指令操作码，向ALU控制单元输出一个2位控制信号ALUOp（ALU操作码）

当ALUOp为00时，表示这条指令是访存指令，需要ALU将基址和偏移量相加（0010）
当ALUOp为01时，表示这条指令是分支指令beq，需要ALU将两源操作数相减（0110）
当ALUOp为10时，表示这是条R型指令，由funct字段进一步指定需要ALU完成的功能

判定为R型指令后，ALU控制单元将6位功能码funct字段映射到
加0010、减0110、与0000、或0001、小于则置位0111中的一个

ALUOp		funct 字段						操作
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
0	1	X	X	X	X	X	X	0110
1	0	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	0	X	X	0	1	0	0	0000
1	0	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

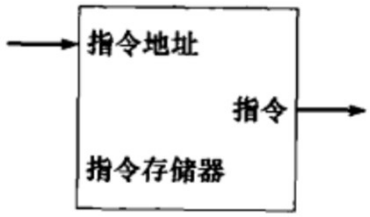
ALU控制线由主控制单元发出的ALUOp、R型指令中的funct字段两级共同决定，这种多级译码方式有利于提高控制器性能

控制信号：指令存储器、寄存器堆、ALU

一个程序在运行前将指令写入指令存储器，不需要专门的写使能控制信号来“批准”写入

指令存储器每周期进行一次读操作，是只读且必读的

不需要专门的读使能控制信号来“批准”读出



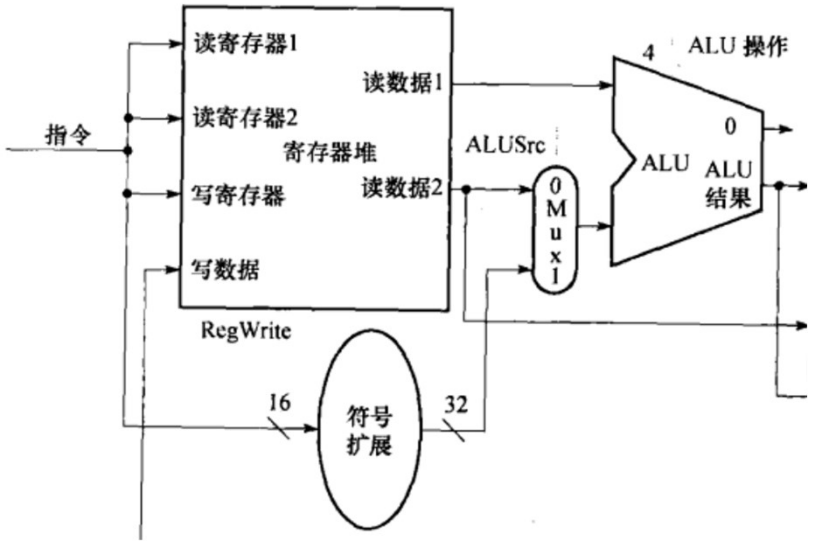
8条MIPS核心子集指令都会读寄存器，无需读使能控制信号

但是，只有R型指令和lw指令会写回寄存器堆

因此要有寄存器堆写使能控制信号RegWrite

并用启用rd控制信号RegDst决定写回rd还是rt

j指令呢？什么时候需要读/写使能控制信号？



对ALU，除了上述两位ALUOp信号，还需要

启用立即数（作为ALU源）控制信号ALUSrc

为1时选取符号扩展后的立即数作为ALU源操作数

为0时选取rt作为ALU源操作数

控制信号：数据存储器、写回

只有lw读数据存储器，只有sw写数据存储器

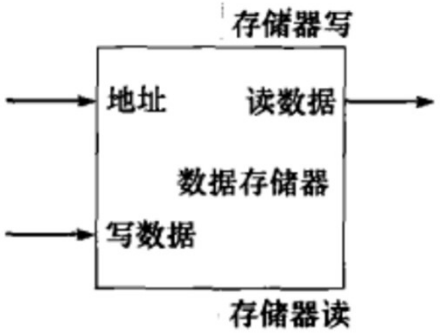
因此，数据存储器需要读、写控制信号各一个：

（数据）存储器读使能控制信号MemRead

（数据）存储器写使能控制信号MemWrite

因为指令存储器不需要读写使能控制信号

不必强调是“数据”存储器



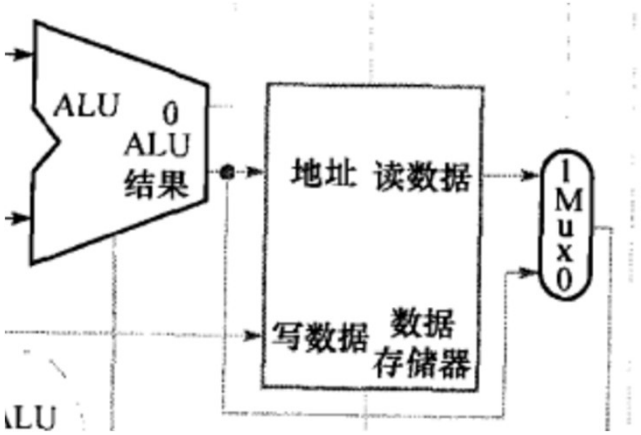
R型指令从ALU写回寄存器，lw从存储器写回寄存器

为了选择写回寄存器的数据来源

需要一个存储器写回控制信号MemtoReg

为1时将存储器数据写回

为0时将ALU运算结果写回



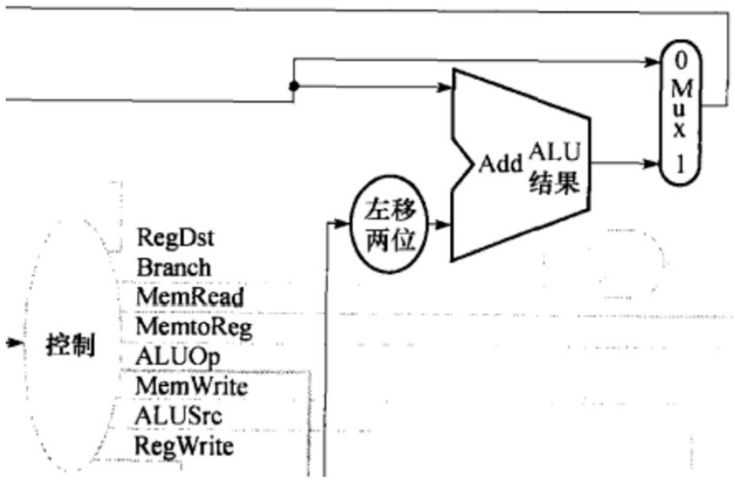
B站 翼云图灵

控制信号：分支

分析分支指令beq使用的控制信号
首先，仅当遇到beq指令才可能分支
其次，分支条件为真即ALU运算rs-rt=0时才分支

使用一个与门（AND gate）进行与运算
当两个输入均为1时，才输出1
当分支控制信号Branch和ALU零标志Zero同时为真
才将PC源控制信号PCSrc置为1
选择将分支目标地址写回PC，完成分支

只要其中一个条件为假（当然包括两个都为假）
选择PC+4写回PC，取消分支



B站 翼云图灵

控制信号小结

指令周期第二阶段ID所谓的“指令译码”

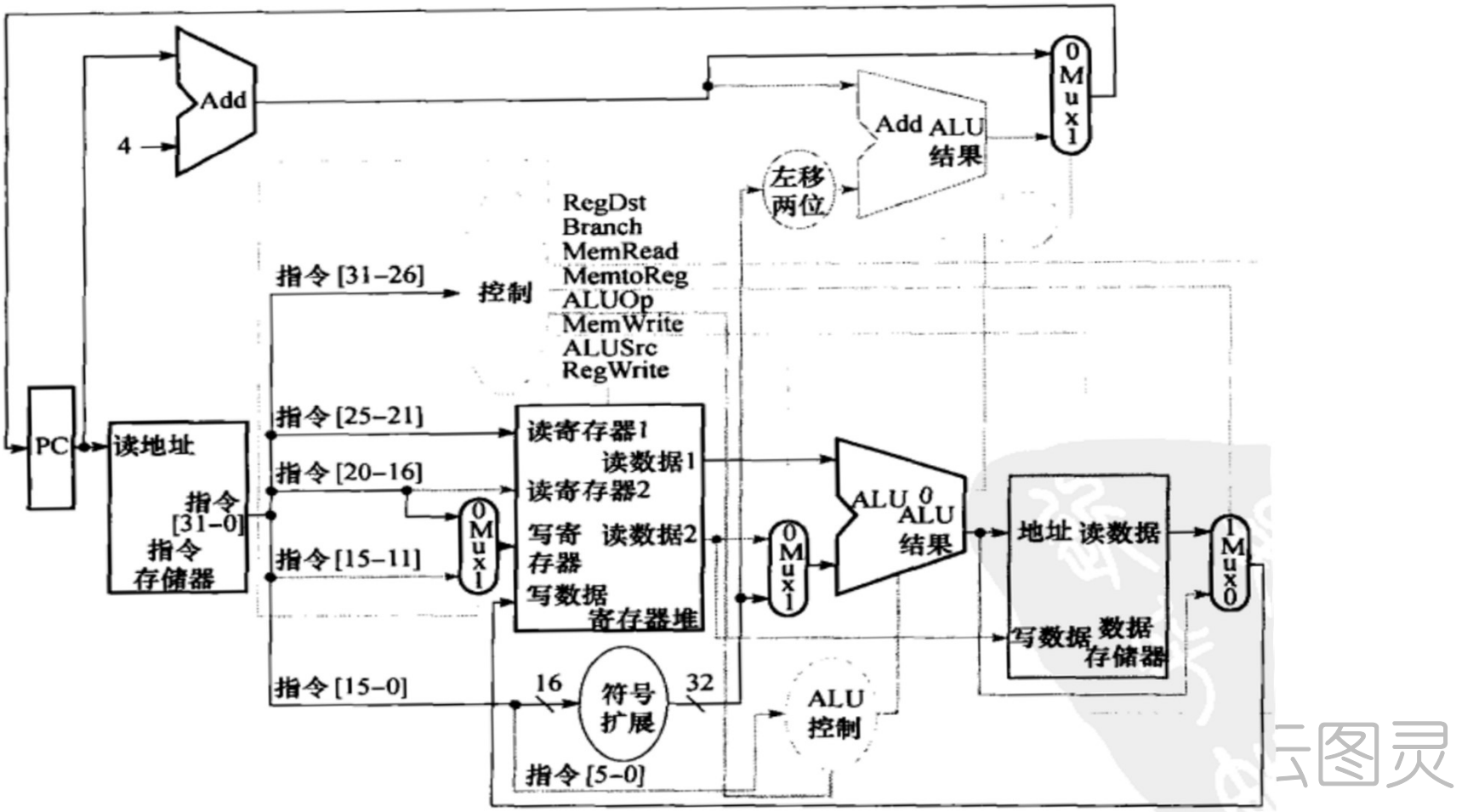
就是主控制单元将操作码（不包括功能码）翻译成控制信号，并发送到对应器件的过程

主控制单元共发出8个（共9位）控制信号，由指令Op字段和R型funct字段译码而得

*注意，控制单元只发出Branch信号，而不直接发出PCSrc信号

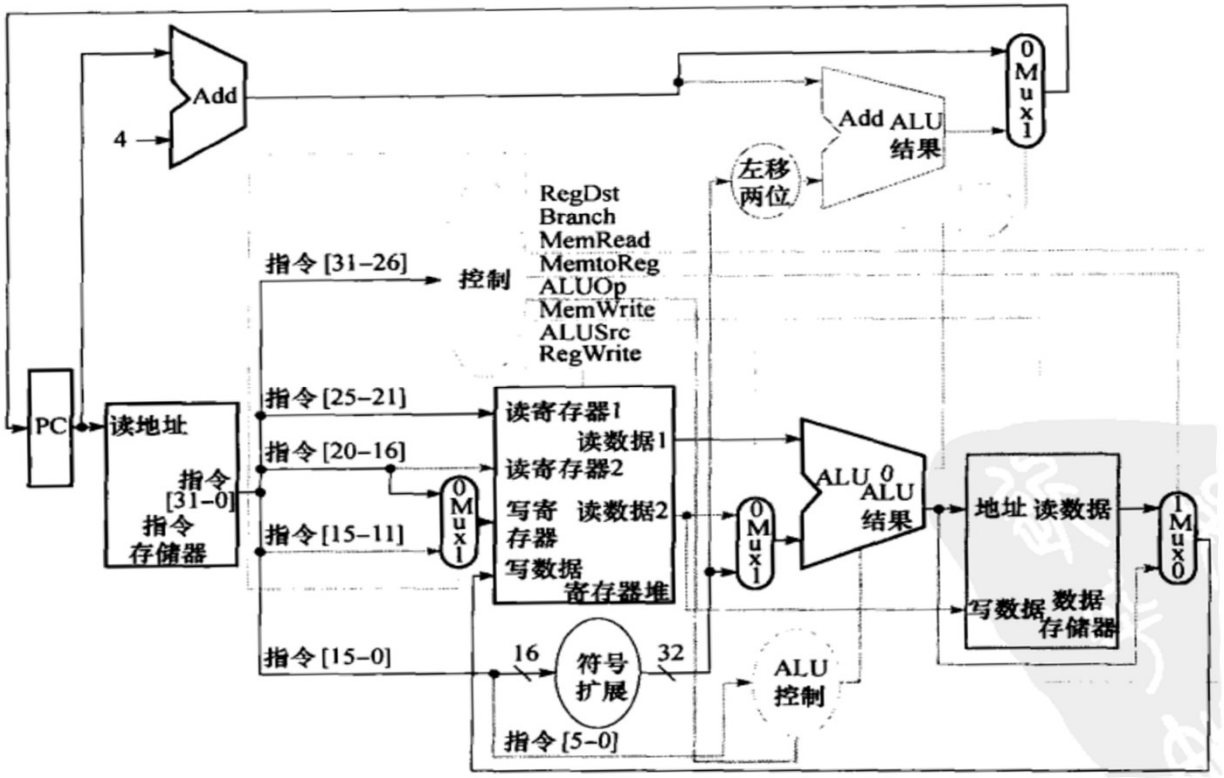
控制信号名	无效时的含义	有效时的含义
RegDst	写寄存器的目标寄存器号来自 rt 字段（位 20:16）	写寄存器的目标寄存器号来自 rd 字段（位 15:11）
RegWrite	无	寄存器堆写使能有效
ALUSrc	第二个 ALU 操作数来自寄存器堆的第二个输出（读数据 2）	第二个 ALU 操作数为指令低 16 位的符号扩展
PCSrc	PC 由 PC +4 取代	PC 由分支目标地址取代
MemRead	无	数据存储器读使能有效
MemWrite	无	数据存储器写使能有效
MemtoReg	写入寄存器的数据来自 ALU	写入寄存器的数据来自数据存储器

带控制的MIPS核心子集数据通路



控制信号分析：lw

以lw $t0, 12($s0)为例

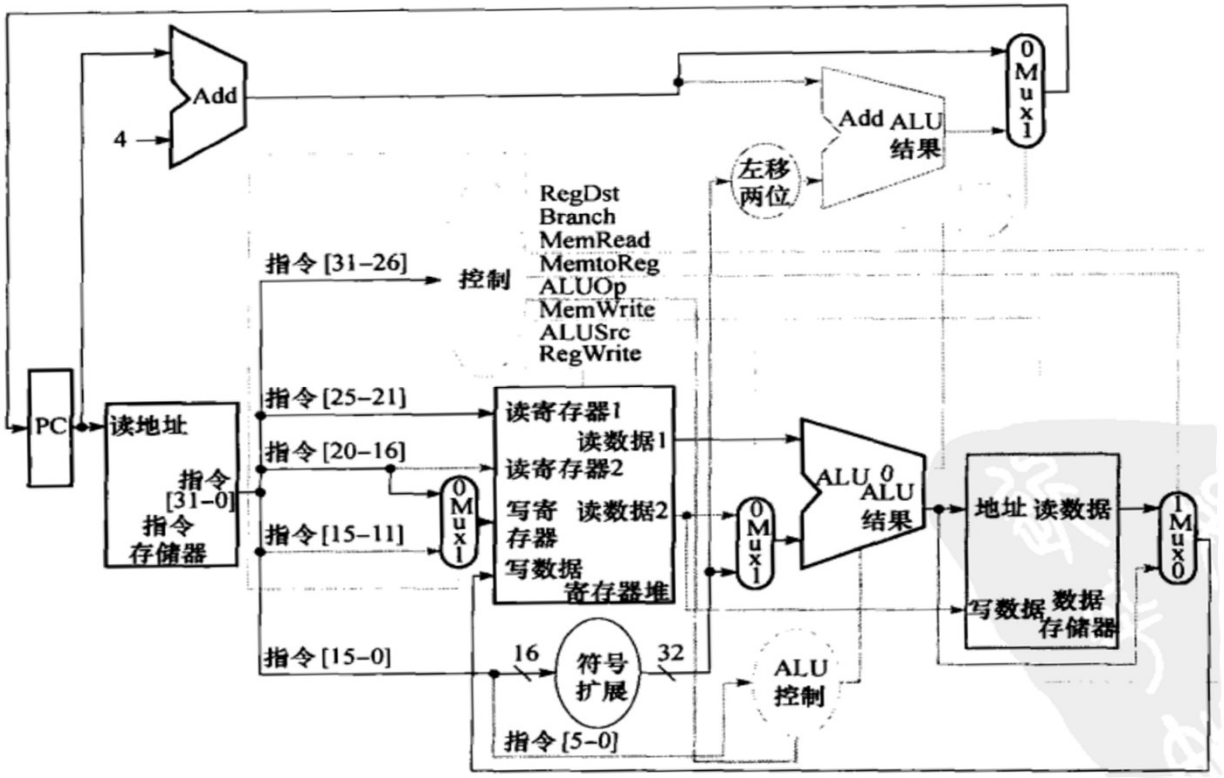


控制信号名称	功能	值
RegDst	启用rd	
ALUSrc	imm输入ALU	
MemtoReg	存储器写回	
RegWrite	写寄存器堆	
MemRead	读存储器	
MemWrite	写存储器	
Branch	分支	
ALUOp1	ALU操作码	
ALUOp0		

B站 翼云图灵

控制信号分析：beq

以beq $t0, $zero, Loop为例，其中$t0=0，Loop的16位PC相对字偏移量为0008H



控制信号名称	功能	值
RegDst	启用rd	
ALUSrc	imm输入ALU	
MemtoReg	存储器写回	
RegWrite	写寄存器堆	
MemRead	读存储器	
MemWrite	写存储器	
Branch	分支	
ALUOp1	ALU操作码	
ALUOp0		

B站 翼云图灵

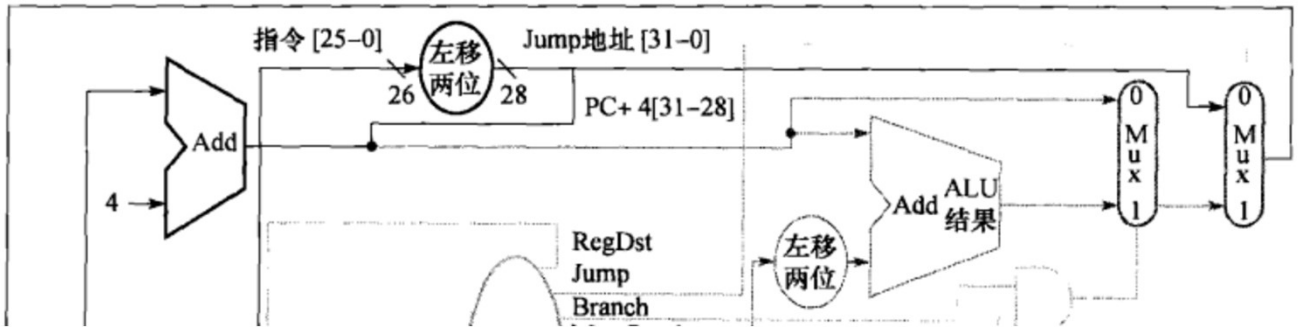
指令控制信号表 j指令扩充

对R型指令、sw的控制信号分析留作复习题

对R型、访存、分支指令，控制信号分别为（X表示无关项）：

*川大期末必考10分的控制信号填表，请务必熟记于心

指令	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
R 型	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1



B站 翼云图灵

复习题

1、组合逻辑和状态/时序逻辑的划分依据是什么？状态单元采用什么时钟方法?

2、选取哪9条指令作为本章实现数据通路的MIPS核心子集?

3、MIPS指令周期包含五个阶段，各阶段的名称、作用、使用部件分别是什么?

4、单周期实现机制为什么性能不佳?

5、说明ALU控制信号的两级译码过程。2位ALUOp和4位ALU控制线有什么区别和联系?

6、什么情况下需要读/写使能控制信号?

7、主控制单元发出哪8个控制信号？各自有什么作用?

8、PCSrc由哪两个信号共同决定?

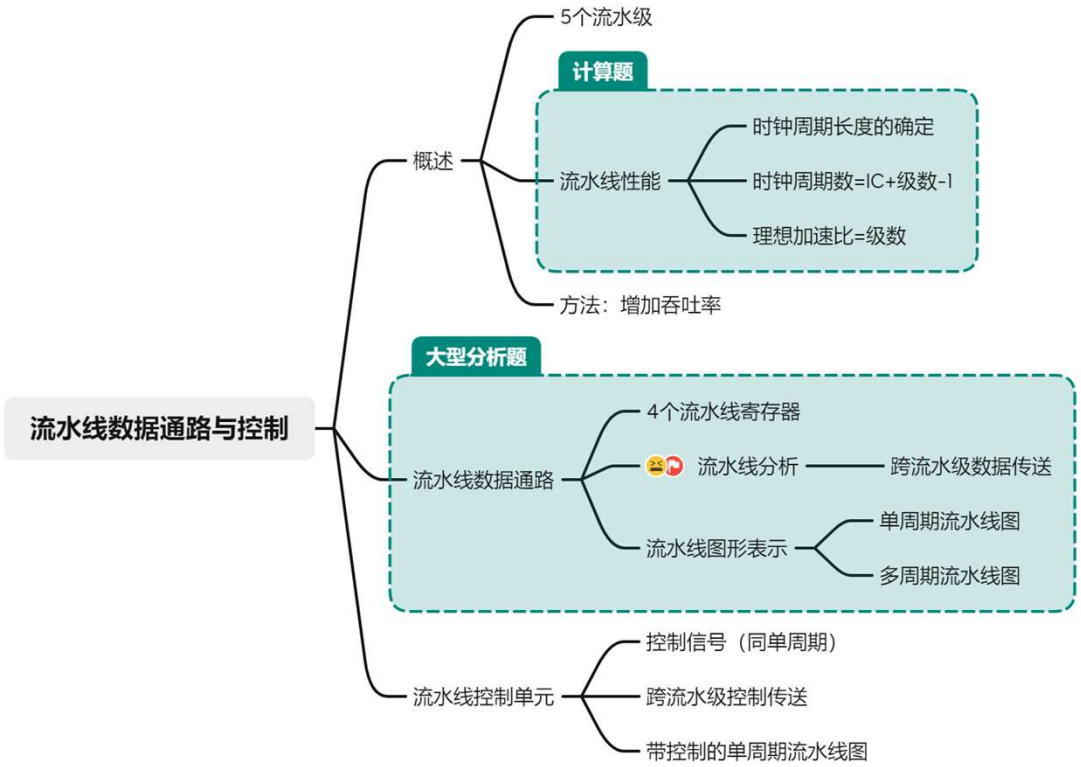
9、自行分析R型指令、sw指令的控制信号，熟悉带控制的数据通路图和指令控制信号表

*10、尽量不翻书，画出带控制的MIPS核心子集数据通路（含j指令），核对P182图4-24

B站 翼云图灵

第三部分

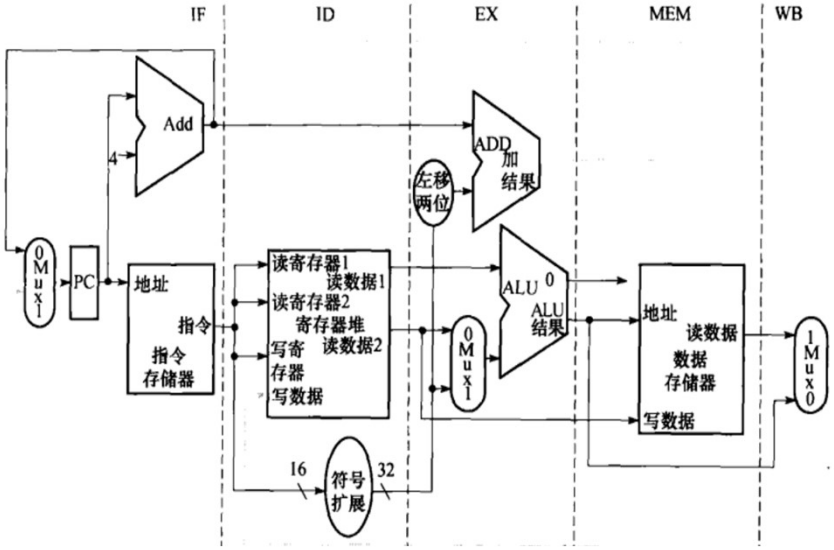
流水线数据通路与控制



B站 翼云图灵

指令周期与流水级

	IF取指令	ID译码与读reg	EX运算	MEM访存分支	WB写回
R型	根据PC 从存储器中 取出指令	取源操作数 rs和rt	执行运算	-	将ALU运算结果 写回rd
lw		取基址寄存器rs 偏移量符号扩展	计算内存地址	读数据存储器 取得数据	将访存数据 写回rt
sw				写数据存储器	-
beq		取rs和rt 字地址符号扩展 并左移两位	比较rs和rt 算分支目标地址	地址写回PC	-



单周期实现中，任一时刻只有部分硬件在运行

将指令执行过程分散在五个周期，每周期只执行一个阶段

指令1进入ID周期后，指令2可以使用IF部分的硬件

指令1进入EX周期后，指令2可以使用ID部分的硬件，指令3可以使用IF部分的硬件……

与指令周期的五个阶段相对应，把数据通路分为五个流水级，形成流水线（pipeline）

在流水线中，可以称XX阶段为XX周期

B站 翼云图灵

流水线时钟周期的长度T和数量cycles

假设五个阶段各需200ps，将最慢阶段的200ps作为时钟周期

和单周期相比，执行4条指令、10000条指令的的加速比分别为多少？

时钟周期数 = 指令数+流水级级数-1　cycles = IC+n-1

理想加速比 = 流水线级数　$S_{n理想} = n$

理想条件为：①每个流水级时间等长；②流水线没有开销；③指令数足够大

假设五个阶段需要的时间分别为200ps、100ps、200ps、200ps、100ps

仍然将最慢阶段的200ps作为时钟周期

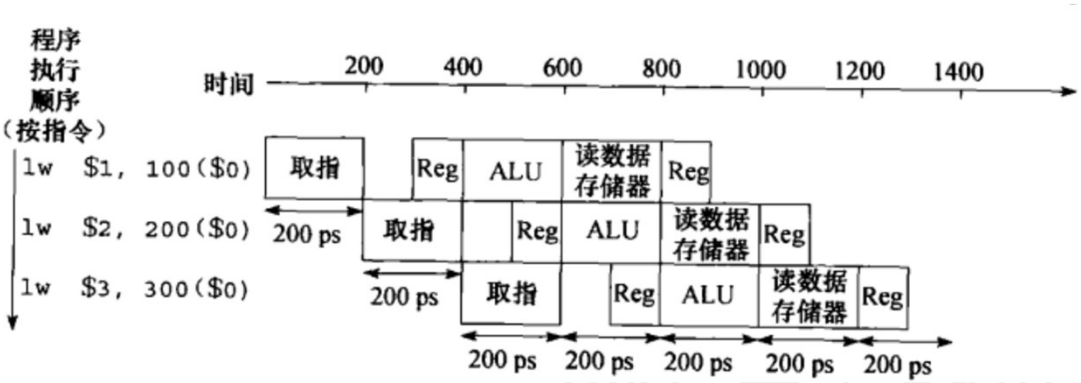
lw后跟一条add指令

能否砍掉add的MEM流水周期？

省略流水周期可能导致

两条指令抢占同一流水级的硬件部件

引发结构冒险（structural hazard）



流水线性能

只观察一条指令，流水线是否改善了它的执行时间？

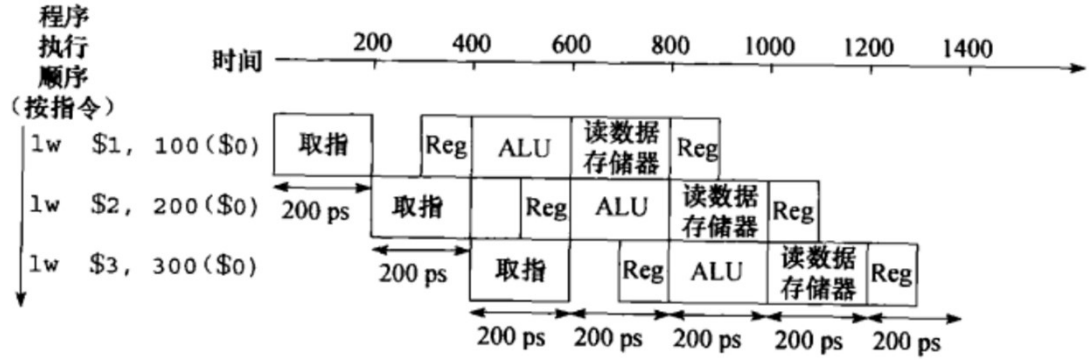
流水线并不减少单条指令的执行时间，而是通过增加指令吞吐率来提高性能

即，在同一时间处理多条指令的不同阶段，实现指令级并行

理想情况下，流水线CPI为1，IC不变

由时钟周期长度决定的吞吐率

是评价流水线性能的重要指标



为提高流水线性能

可进一步划分流水级、缩短时钟周期

在IC和CPI不变的情况下，进一步缩短T，减少CPU执行时间，提高吞吐率

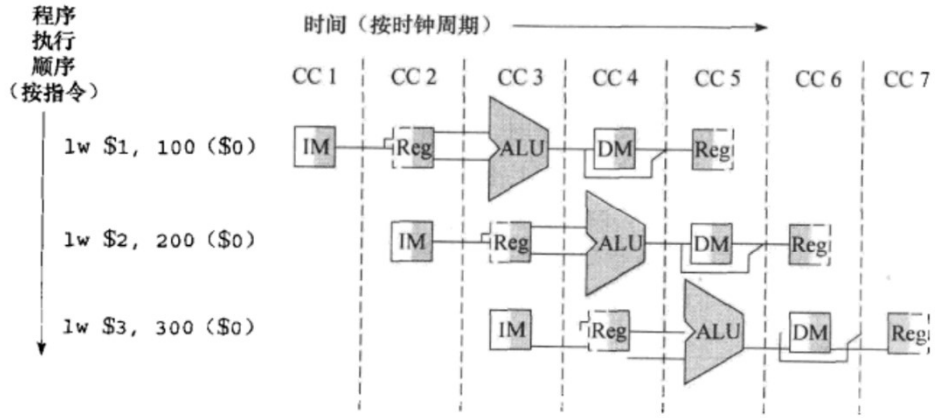
*过度划分流水级会导致调度开销增大、分支性能下降，抵消性能提升

MIPS是一种精心为流水线设计的指令集，很容易避免结构冒险

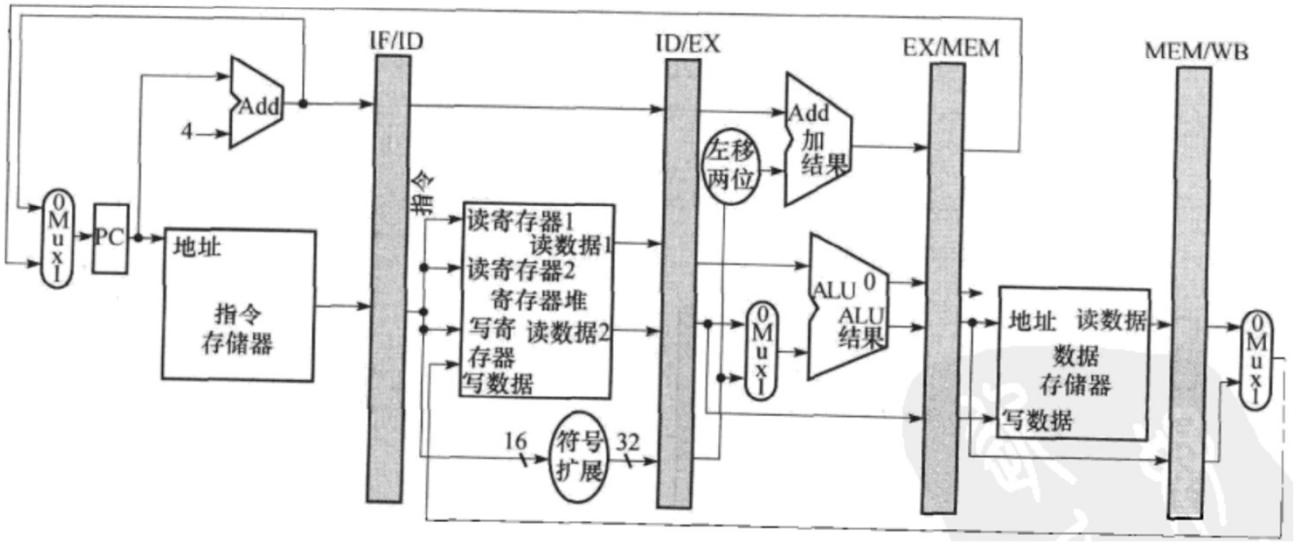
B站 翼云图灵

流水线寄存器

任一时刻，每个流水级只被一条指令占用
用一条数据通路执行5条MIPS指令
不会造成结构冒险



但每条指令使用和生成的数据各不相同
为了保留指令各自的数据
需要在两个流水级之间
插入流水线寄存器
以左右两个流水级命名
分别叫做IF/ID、ID/EX、
EX/MEM、MEM/WB



流水线分析：lw

约定状态/时序单元左半边涂灰表示写入，右半边涂灰表示读取；组合单元涂灰表示使用

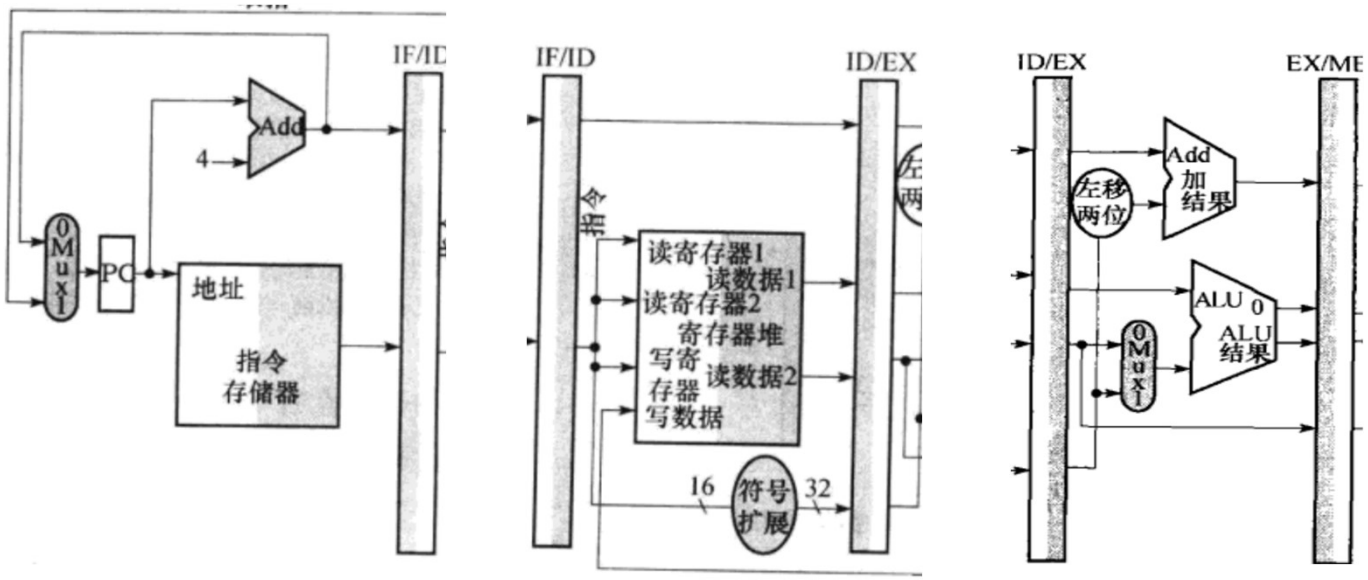
IF：一切指令都要取指令，将PC+4和指令传给IF/ID

ID：一切指令都要译码产生控制信号，除j以外均要读取寄存器

PC+4继续传给ID/EX，rs和rt的数据、扩展后的立即数也要传给ID/EX

EX：多选器根据ALUSrc选择ALU源操作数和rs相加

ALU结果、Zero标志位、分支目标地址都要传给EX/MEM

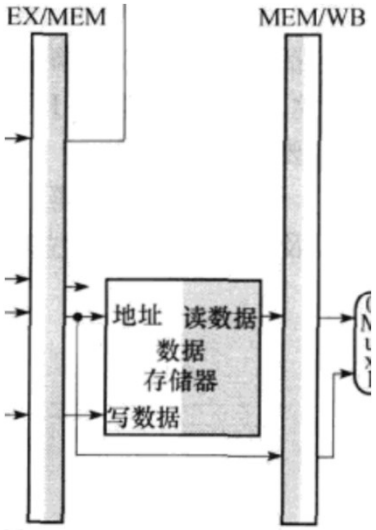


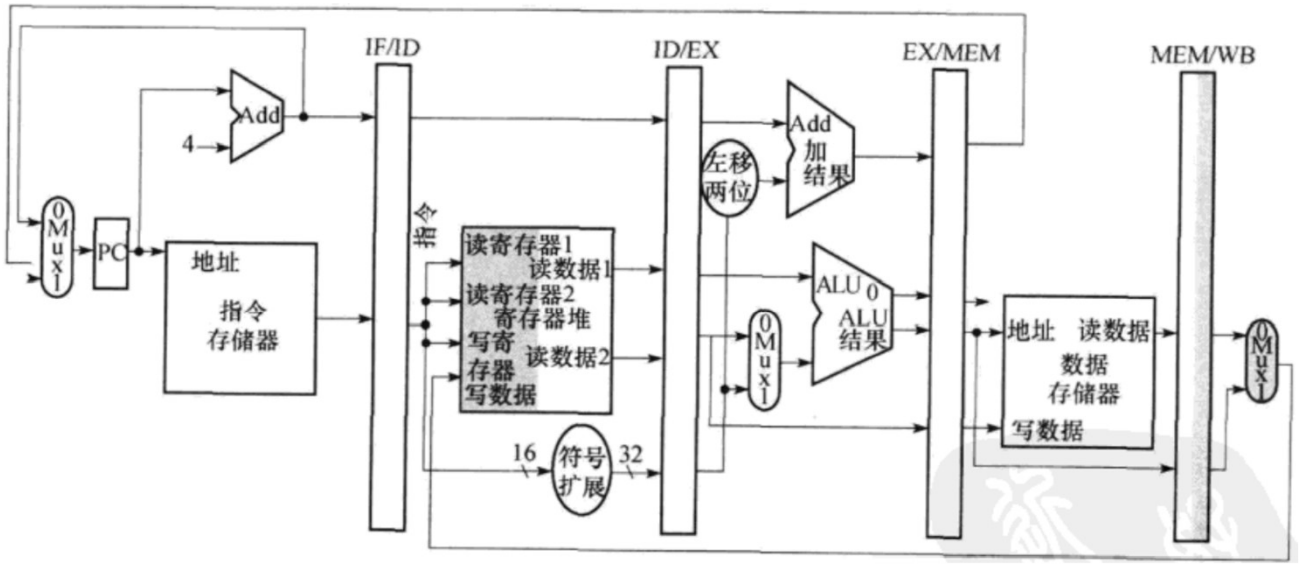
B站 翼云图灵

流水线分析：lw

MEM：从数据存储器中读取数据， 和ALU运算结果一并传给MEM/WB

WB：将访存读取的数据写回寄存器堆 为了写回rt，我们缺少什么信息？



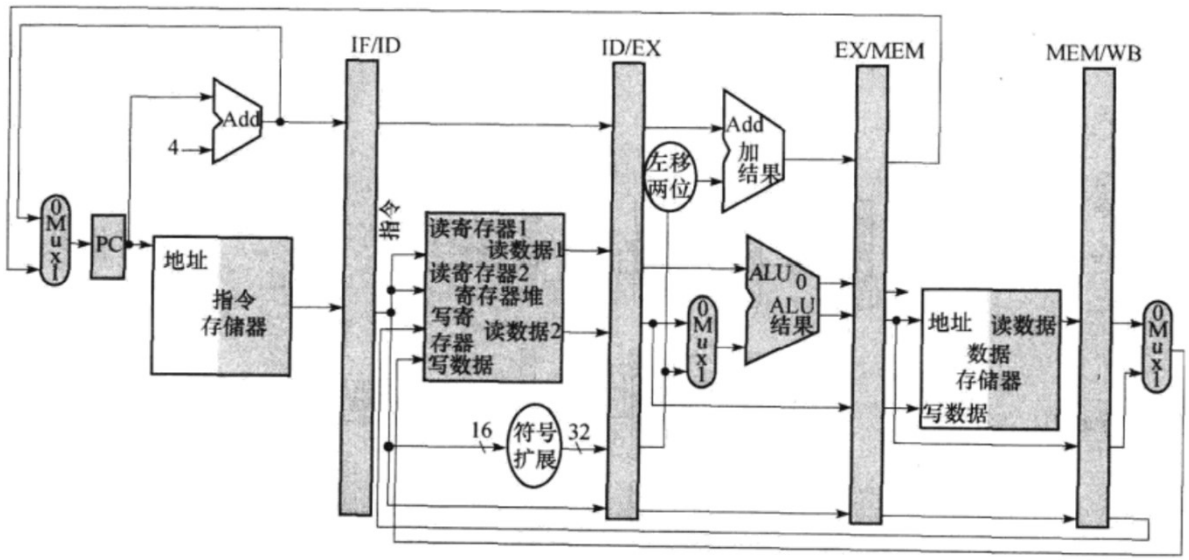


B站 翼云图灵

流水线分析：跨流水级数据传送

为确定lw写回哪个寄存器，rt的寄存器号要从ID级一直传递到WB级

进行跨流水级的数据传送，改进后的流水线数据通路如下图（所有使用过的部件涂灰）：



在lw第三阶段的图中，传递rt的内容用意何在？

对sw的流水线分析留作复习题

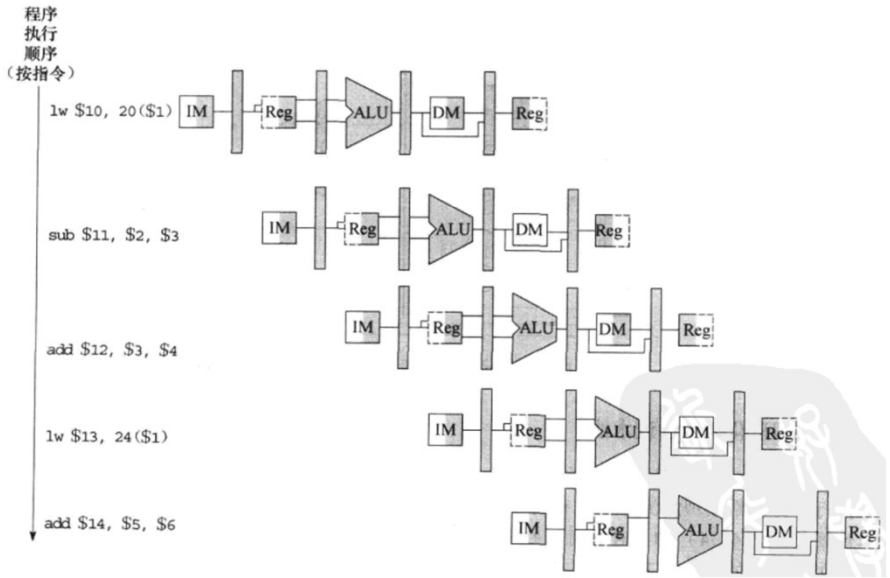
B站 翼云图灵

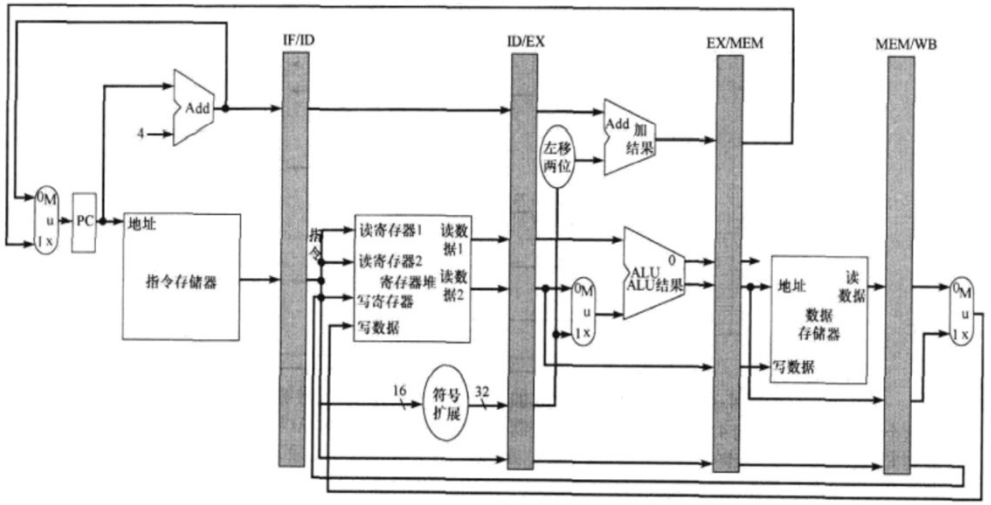
流水线图

多周期流水线图从总体上描述流水线指令序列

可以转化为一系列单周期流水线图，刻画同一周期内五个流水级的工作情况

会认读并草稿作图即可，不必纠结细节





B站 翼云图灵

流水线控制

流水线控制信号和单周期控制信号的种类、数量、功能完全相同

教材P204对流水线控制信号按流水级重新进行了排序，含义完全没变

但是每条指令在ID级产生控制信号后，下一周期会被后续指令覆盖

考虑指令序列add-lw-sw

add从EX级进入MEM级以后

lw进入EX级，但EX中的控制信号还是add的信号

add也需要把自己的信号移动到MEM级之中

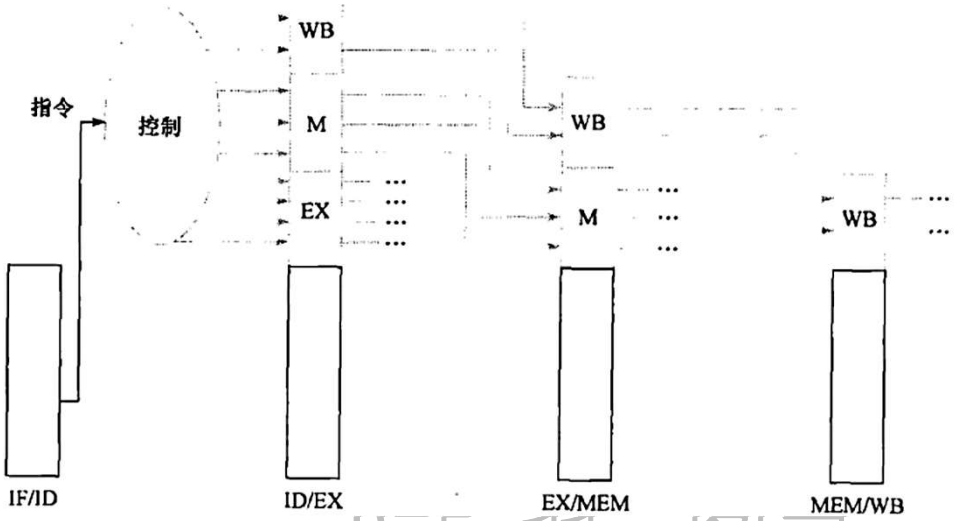
因此，控制信号也需要从ID级依次传到

EX级、MEM级、WB级

EX级使用该级的信号（ALUSrc和ALUOp）

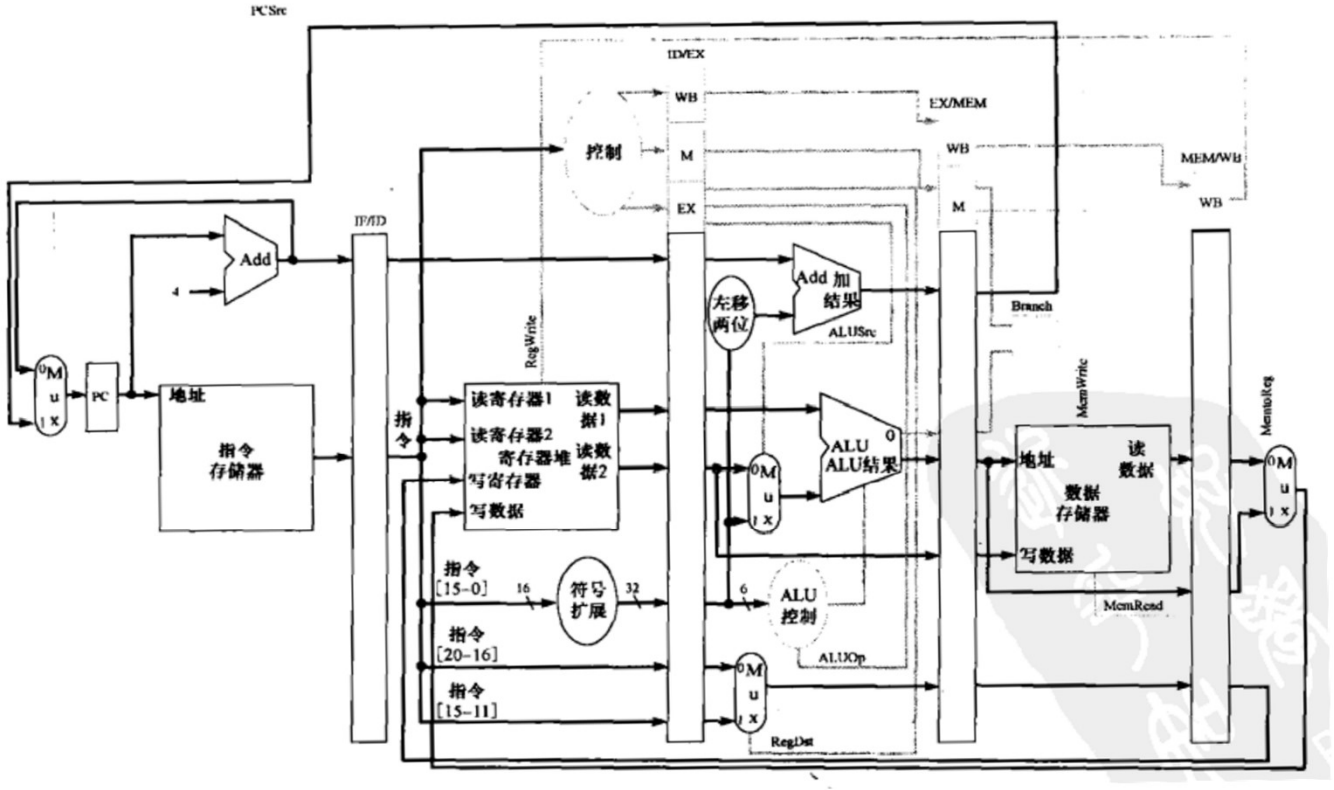
后不再使用，可以丢弃，MEM级同理

这就是跨流水级的控制信号传送



B站 翼云图灵

带控制的流水线数据通路 指令控制信号表



指令	执行/地址计算阶段的控制信号				存储器存取阶段的控制信号			写回阶段的控制信号	
	RegDst	ALUOp1	ALUOp0	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
R 型	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

注意EX级最下面的多选器

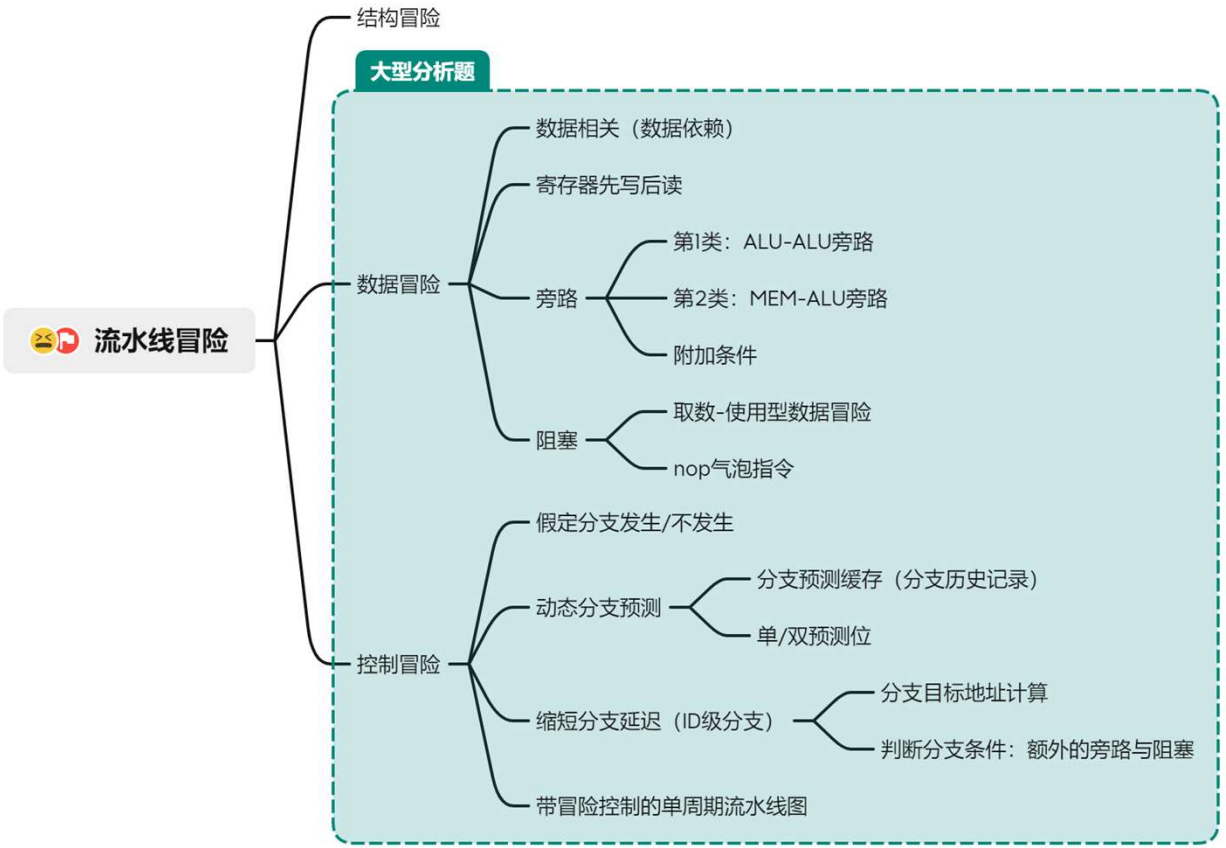
其作用是什么？

EX级实际有几个控制信号？

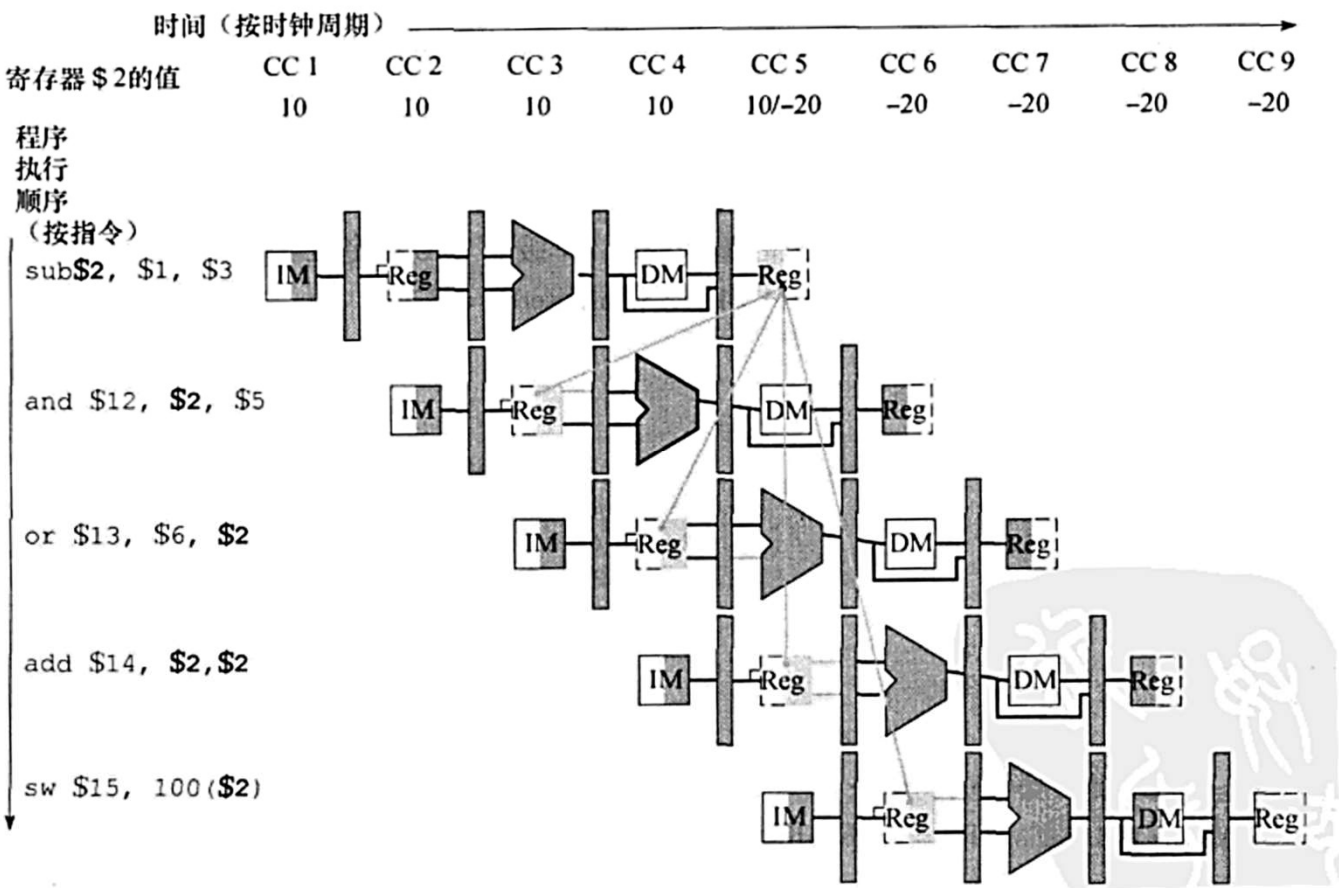
B站 翼云图灵

第四部分

流水线冒险


流水线冒险
结构冒险
大型分析题
数据冒险
数据相关（数据依赖）
寄存器先写后读
旁路
第1类：ALU-ALU旁路
第2类：MEM-ALU旁路
附加条件
阻塞
取数-使用型数据冒险
nop气泡指令
控制冒险
假定分支发生/不发生
动态分支预测
分支预测缓存（分支历史记录）
单/双预测位
缩短分支延迟（ID级分支）
分支目标地址计算
判断分支条件：额外的旁路与阻塞
带冒险控制的单周期流水线图

数据相关与数据冒险 寄存器先写后读



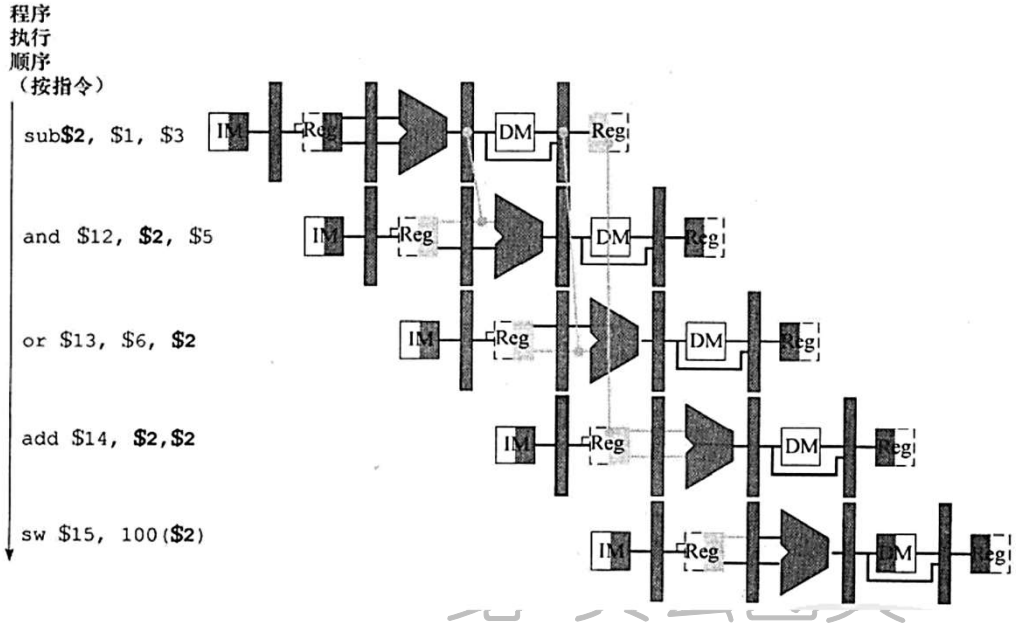
$v0寄存器（$2）被五条指令使用
其中，sub指令将结果-20写入$v0
另外4条指令预期读取数据-20
与sub指令数据相关

但是，sub指令在CC5
才将-20写入$v0
and和or在CC3/CC4就要读取-20
此时$v0的值还是10，于是
产生数据冒险（data hazard)

约定寄存器前半拍写，后半拍读
在CC5中，sub先写结果-20
再由add读取，可避免一次冒险

ALU-ALU旁路 MEM-ALU旁路

实际上，and和or要使用的数据-20在CC3就已经由ALU计算生成
我们可以从EX/MEM寄存器将数据直接传给and指令的ALU
从MEM/WB寄存器将数据传给or指令的ALU
这种跳过寄存器写回、直接从流水线寄存器取得数据的方法
称为转发（forward）或旁路（bypass）



其中，and指令将sub的ALU运算结果
作为ALU的rs输入，形成ALU-ALU旁路

1a. EX/MEM.RegisterRd = ID/EX.RegisterRs

or指令将sub的ALU运算结果（MEM级）
作为ALU的rt输入，称为MEM-ALU旁路
写出另外三个旁路条件表达式
配备两种旁路，则称为全旁路

旁路条件

只有写回寄存器的指令（R型和lw）才能向后续指令发出旁路

第一个附加条件：RegWrite = 1

此外，写回的寄存器不能是$zero

第二个附加条件：EX/MEM.RegisterRd != 0

```
if(EX/MEM.RegWrite
and(EX/MEM.RegisterRd≠0)
and(EX/MEM.RegisterRd = ID/EX.RegisterRs))
ForwardA = 10
```

ALU-ALU旁路条件的三个条件为：

①RegWrite = 1

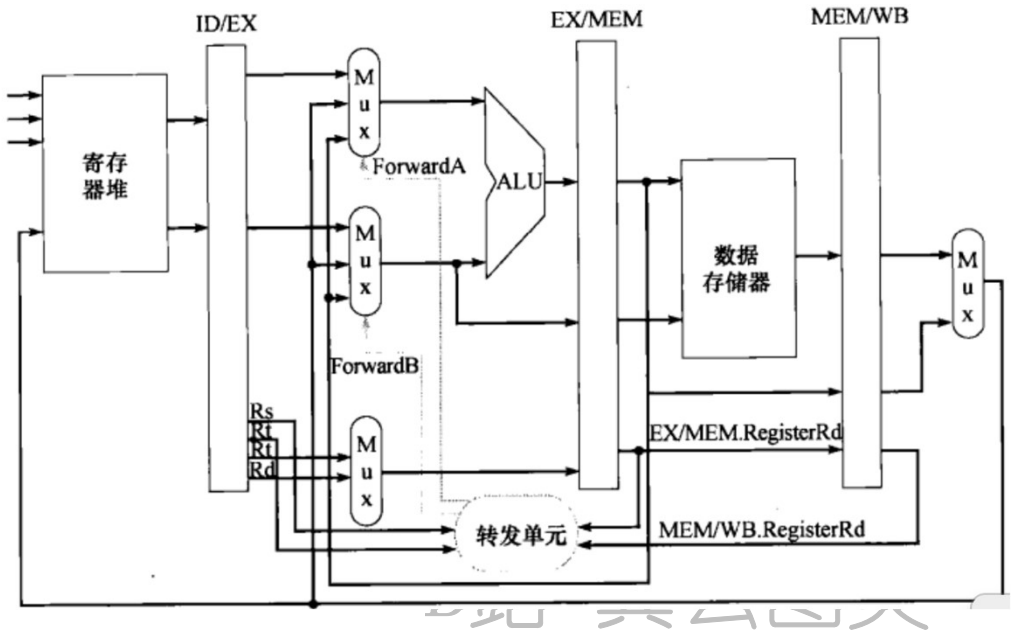
②EX/MEM.RegisterRd != 0

③EX/MEM.RegisterRd = ID/EX.RegisterRs

通过将多选器的控制信号置为10，旁路到rs

即旁路单元设置ForwardA = 10

MEM-ALU旁路条件有四条，增加的条件是!(ALU-ALU旁路)



取数-使用型数据冒险 阻塞

考虑指令序列：

lw $t0, 12($s0)　add $t1, $t0, $t0

能否通过旁路机制解决这一数据冒险？

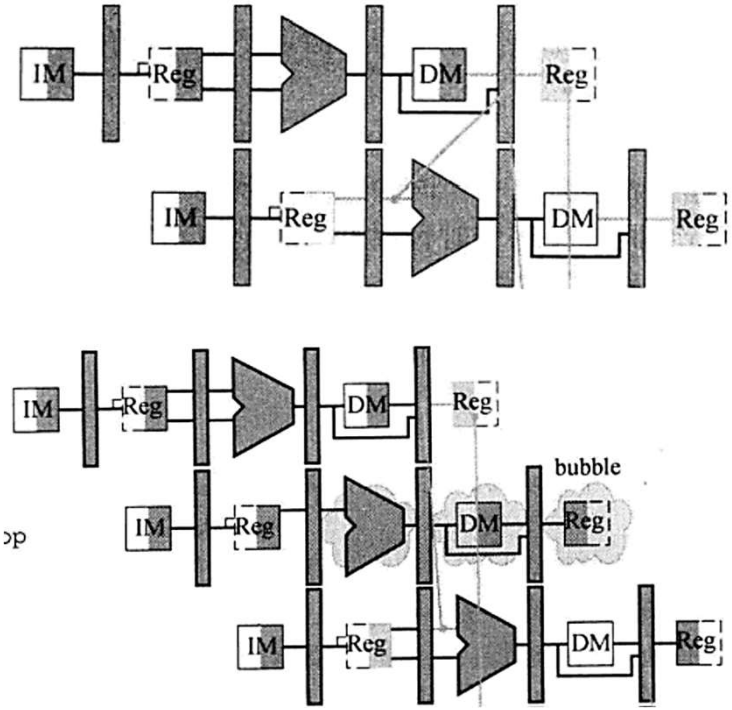
下一条指令使用lw的目标寄存器rt时

访存读出的数据在MEM级才产生

尝试添加旁路，发现数据流向和时间相反

对这种取数-使用型数据冒险

必须在lw指令后添加一个气泡/阻塞周期



add指令已经执行IF和ID周期，我们并不是真的塞入一条空指令nop

而是把add的控制信号全部清零、使其不改变任何状态单元，将其变为空指令

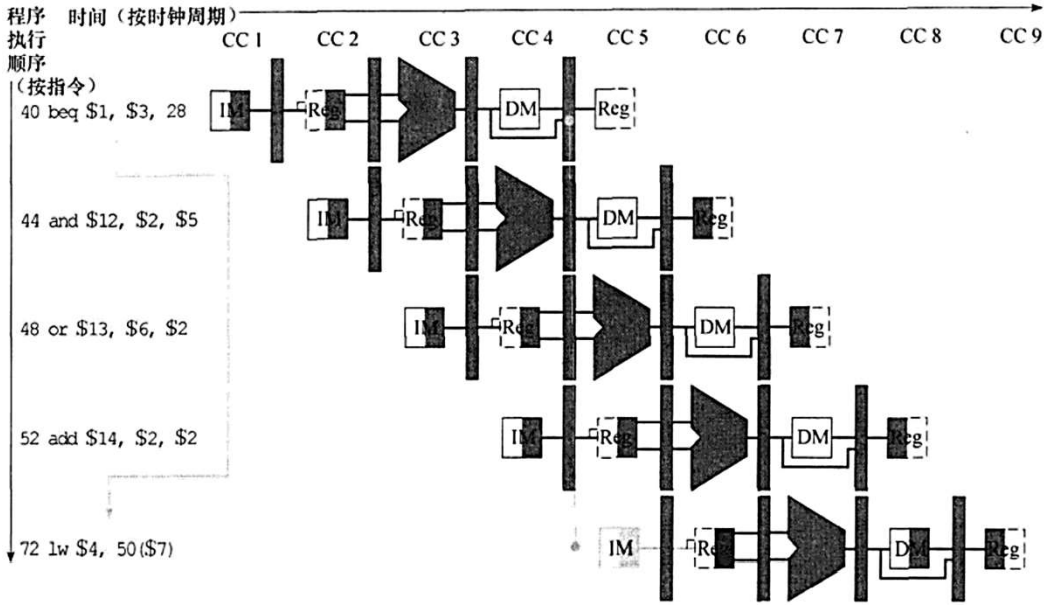
同时将add指令的地址（PC+4-4）重新写回PC，在lw的EX周期重新执行add指令

阻塞由冒险检测单元控制实现

B站 翼云图灵

分支引发的控制冒险

分支指令beq在MEM级才能决定是否分支
此前beq后的三条指令都已被取到流水线
分支成功却执行了
分支失败才应该执行的指令（或反过来）
就产生了分支冒险，是一种典型的
控制冒险（control hazard）



当我们猜测每个分支语句发生的概率很小
如循环中用于退出循环的
beq reg1, $zero, Exit指令
我们可以采取总是假设分支不发生的策略，执行那些紧跟在beq后的指令，跳过分支
如果预测失败不产生额外开销，且预测成功率能达到50%，就能减少50%的分支冒险开销
与假设分支不发生相对应，也有总是假设分支发生的策略，不作讨论

B站 翼云图灵

分支预测

当我们猜测每个分支语句发生的概率很小，如循环中用于退出循环的beq reg1, $zero, Exit

我们可以采取总是假设分支不发生的静态分支预测策略

执行那些紧跟在beq后的指令，跳过分支

如果预测失败不产生额外开销，且预测成功率能达到50%，就能减少50%的分支冒险开销

与假设分支不发生相对应，也有总是假设分支发生的策略，不作讨论

通过向数据通路添加称为分支预测缓存（分支历史记录）的小型索引存储器区

保存近几次分支的记录，进而预测分支是否发生，这样的策略称为动态分支预测

动态分支预测的准确率通常高达90%

考虑一个循环，进行9次循环分支（分支发生）后退出（分支不发生）

使用单预测位（初始化为表示不分支的0）会产生两次预测错误：

进入第1次循环后，要继续循环分支，然而预测位为0，预测不分支，产生一次预测错误

进入第9次循环后，不再循环分支，然而预测位为1，预测分支，产生第二次预测错误

B站 翼云图灵

双预测位动态分支预测

使用双预测位时，则只会产生一次预测错误

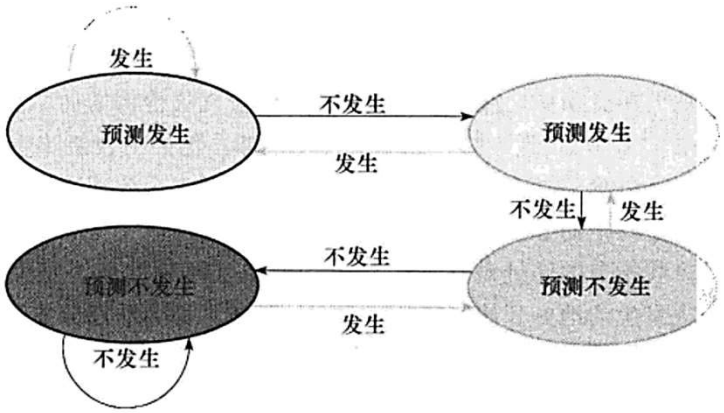
两个预测位表示0~3这4个数，初始化为0

分支一次+1且不超过3，不分支一次-1且不超过0

当预测位为0、1时，预测分支不发生

当预测位为2、3时，预测分支发生

形成右边的有限状态机



过去曾采用延迟分支策略，随着集成度的提高，已经被动态分支预测取代

后来产生了相关预测、竞争预测等新的分支预测方法

小测验

考虑三种分支预测机制：预测分支不发生、预测分支发生和动态分支预测。假定它们在预测正确时无开销，预测错误时开销为两个时钟周期，动态预测器的平均准确率为90%。在此情况下，对下面的分支而言哪种预测器是最好的选择？

A. 分支发生概率为5%。

B. 分支发生概率为95%。

C. 分支发生概率为70%。

B站 翼云图灵

缩短分支延迟

在此前的分析中，beq在MEM级才能决定是否分支
如果能将整个分支过程提前到EX级，
就能将分支错误时的开销从3条指令（3个周期）缩短到2条指令（2个周期）
提前到ID级，就能将分支错误开销进一步降低到1条指令

这种缩短分支延迟的策略需要提前2件事：
①计算分支目标地址、②判断分支条件
PC+4的值在IF周期已经计算出来，完全可以在ID周期计算出分支目标地址
在ID级从寄存器堆取出rs和rt的数据
送入一个相等检测单元
判断相等以后，即可向控制PCSrc的多选器发出信号
将分支目标地址写回PC，在ID级完成分支

B站 翼云图灵

缩短分支延迟

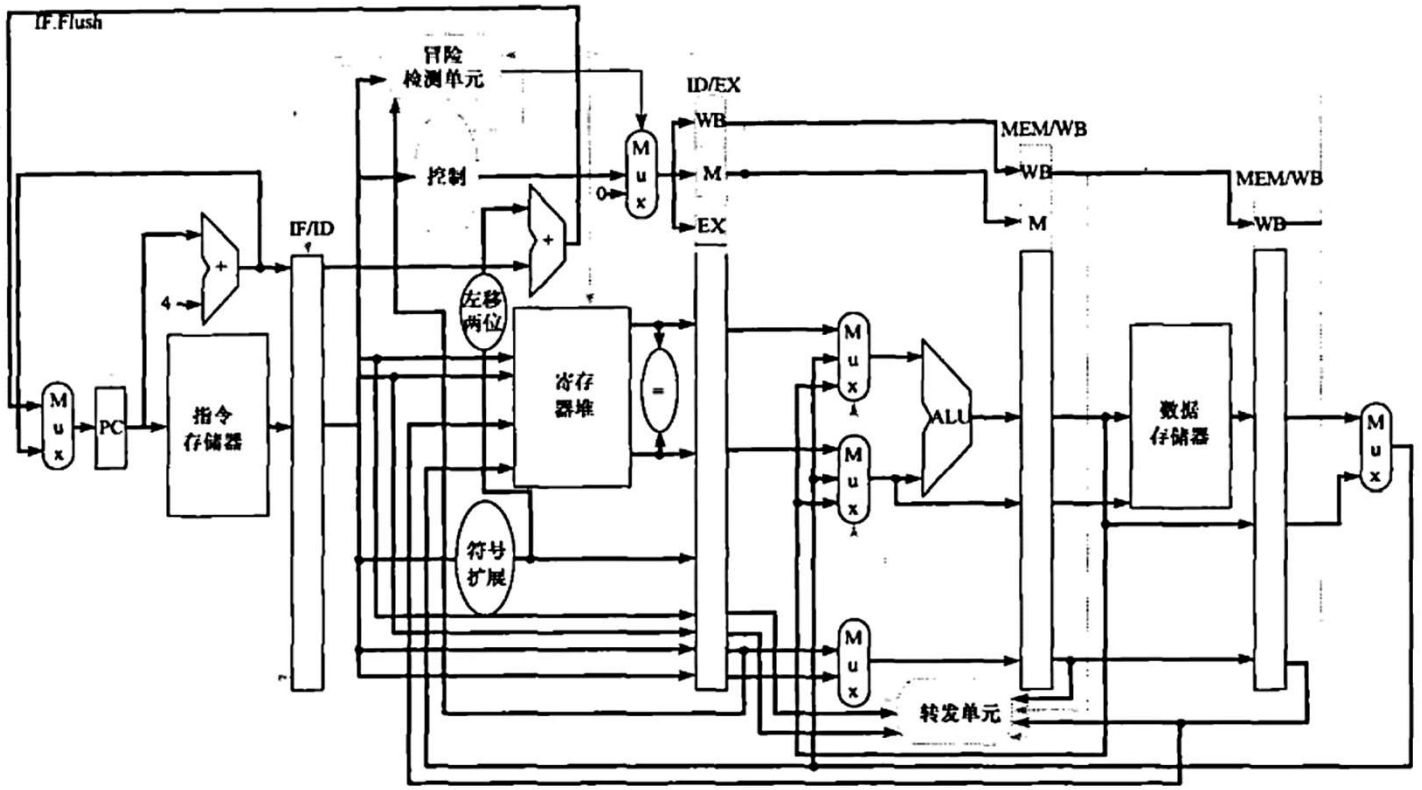
如果beq的上一条指令是R型指令，且需要比较R型指令的运算结果
则，在将R型ALU结果旁路到IF/ID寄存器的基础上
还要将beq指令阻塞1个周期
分析：如果beq比较上一条lw指令取得的数据，需要阻塞2个周期

分支在ID级完成，作出多周期流水线图说明如下指令序列的执行情况：

```
40  beq $1,$3,7
44  and $12,$2,$5
48  or $13,$2,$6
52  add $14,$4,$2
56  slt $15,$6,$7
.....
72  lw $4,50($7)
```

B站 翼云图灵

带冒险控制的单周期流水线图



观察理解：

1）支持旁路的旁路单元

2）支持阻塞的冒险检测单元

3）用于在ID级比较两数是否相等的相等检测单元

全图已经过于复杂，省略了一些之前补充过的细节，不必深究

B站 翼云图灵

异常 x86和ARM处理器 指令级并行加速

异常（中断）是另一种控制冒险

异常源于指令本身，如算术溢出、请求系统调用、使用未定义的指令

中断源于指令外部，如I/O请求、硬件故障（部分硬件故障也属于异常）

为了保存发生异常的地址和原因，需要向数据通路添加异常程计数器EPC和Cause寄存器

ARM Cortex-A8采用三阶段、14级的流水线结构

Intel Core i7 920采用更复杂的流水线，通过将x86指令翻译成微操作来简化流水线实现

流水线和多发射都通过重叠执行多条指令，来开发指令级并行

多发射由编译器实现时称为静态多发射（超长指令字VLIW）

由硬件实现时称为动态多发射（超标量）

受限于功耗，现在的处理器一般采用相对简单的流水线

利用指令级并行循环展开CH3改进后的矩阵乘法程序，实现了性能翻倍

B站 翼云图灵

复习题

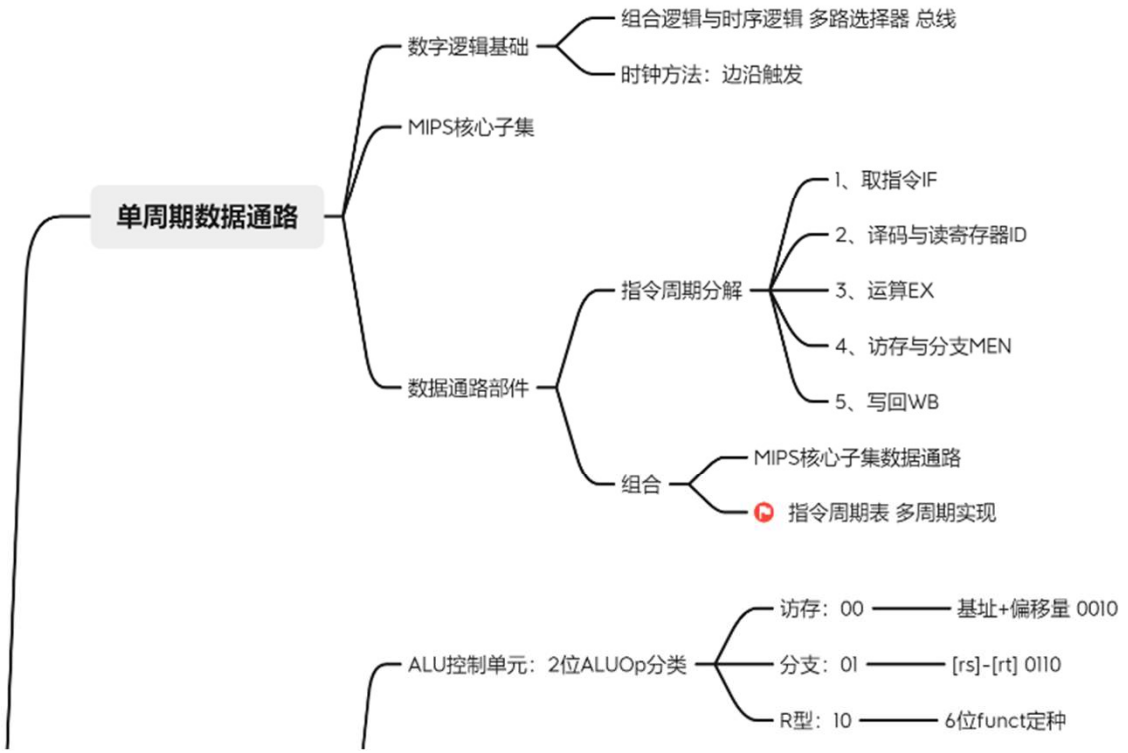
1、解释5个流水级和4个流水线寄存器的关系
2、流水线中的时钟周期长度由什么决定？如何计算流水线时钟周期数？
3、在什么样的理想情况下，流水线的加速比趋近于流水线级数（也叫流水线深度）？
4、流水线的吞吐率由什么决定？如何提高吞吐率？
5、在流水线上执行lw指令，什么信息需要从ID级一直保留到WB级？
6、对照P196~200（注意图文关系，中文版比较混乱），对sw进行流水线分析
7、流水线中的控制信号在何处产生？在逐级传送的过程中会不会减少？
8、如何读写寄存器堆，可以减少一次数据冒险？
9、ALU-ALU旁路和全旁路有什么区别？
10、在什么情况下，需要通过阻塞来解决数据冒险？
11、动态分支根据什么来预测分支是否发生？为什么单预测位机制会导致两次预测错误？
12、为了缩短分支延迟，在ID级添加了什么部件？此时beq指令是否会被阻塞？

B站 翼云图灵

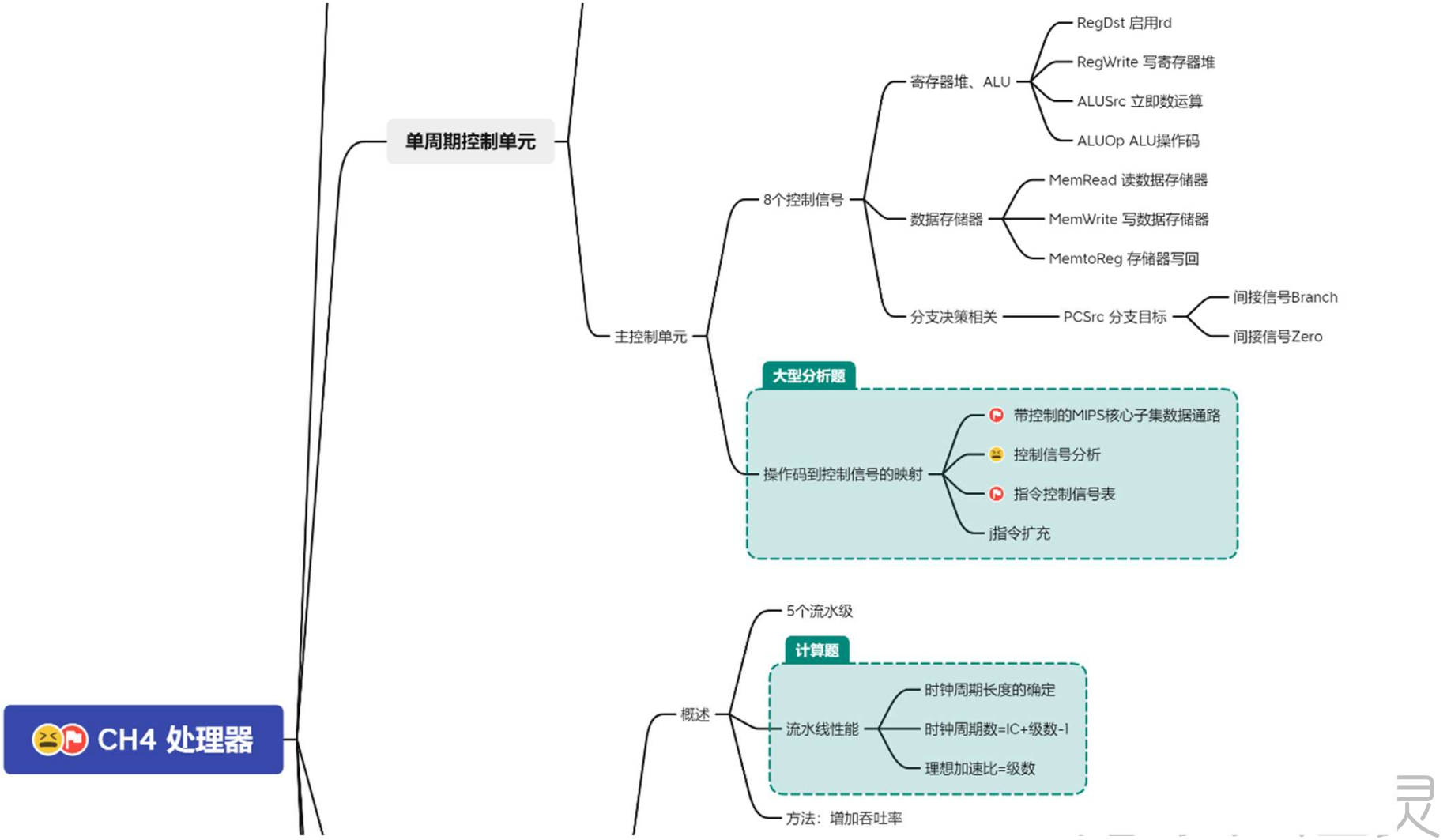
全章复习

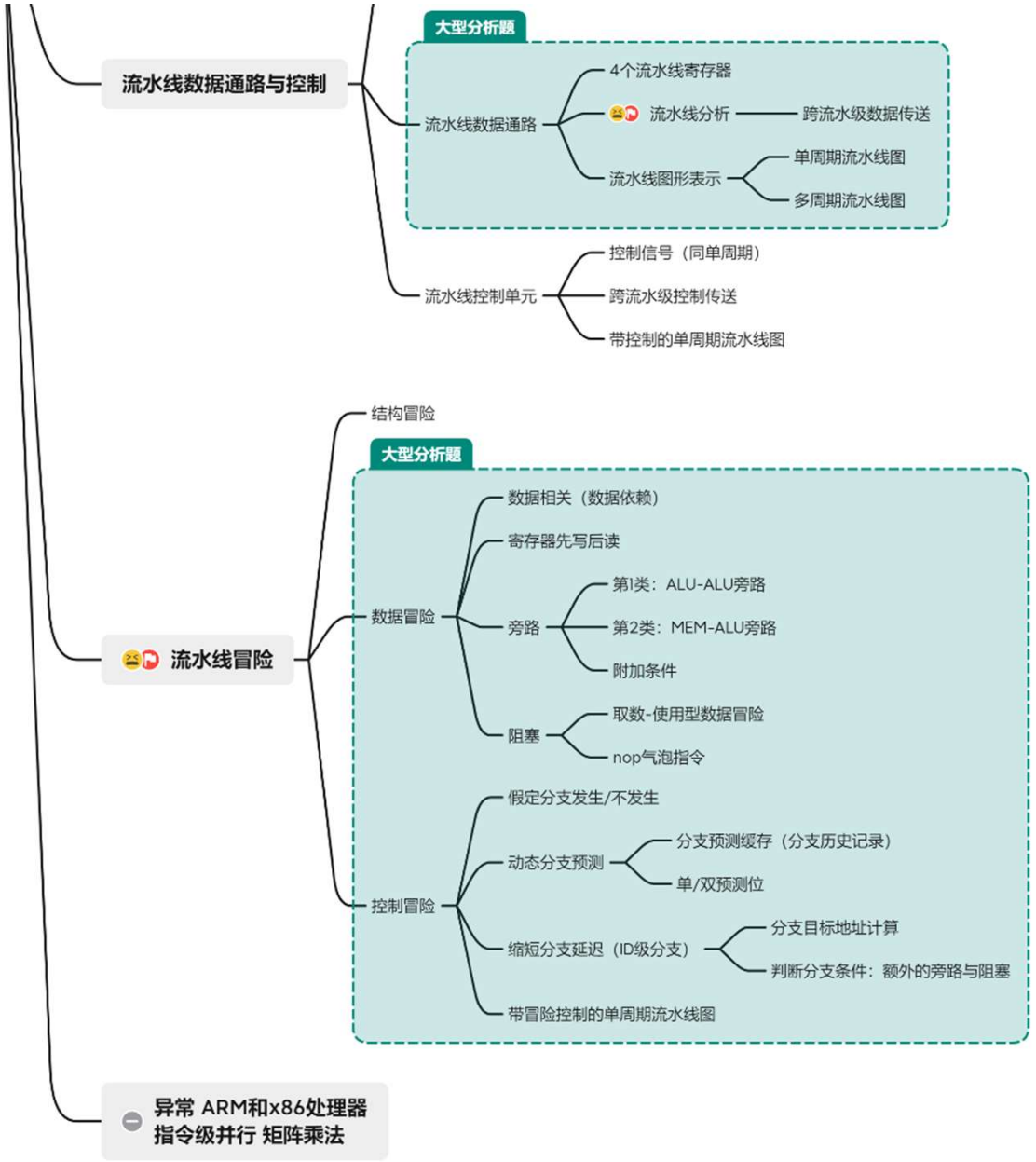
CH4 处理器

B站 翼云图灵


单周期数据通路
数字逻辑基础
组合逻辑与时序逻辑 多路选择器 总线
时钟方法：边沿触发
MIPS核心子集
数据通路部件
指令周期分解
1、取指令IF
2、译码与读寄存器ID
3、运算EX
4、访存与分支MEN
5、写回WB
组合
MIPS核心子集数据通路
指令周期表 多周期实现
ALU控制单元：2位ALUOp分类
访存：00
基址+偏移量 0010
分支：01
[rs]-[rt] 0110
R型：10
6位funct定种

B站 翼云图灵


CH4 处理器
单周期控制单元
主控制单元
8个控制信号
寄存器堆、ALU
RegDst 启用rd
RegWrite 写寄存器堆
ALUSrc 立即数运算
ALUOp ALU操作码
数据存储器
MemRead 读数据存储器
MemWrite 写数据存储器
MemtoReg 存储器写回
分支决策相关
PCSrc 分支目标
间接信号Branch
间接信号Zero
大型分析题
操作码到控制信号的映射
带控制的MIPS核心子集数据通路
控制信号分析
指令控制信号表
指令扩充
概述
5个流水级
计算题
流水线性能
时钟周期长度的确定
时钟周期数=IC+级数-1
理想加速比=级数
方法：增加吞吐率


流水线数据通路与控制
大型分析题
流水线数据通路
4个流水线寄存器
流水线分析
跨流水级数据传送
流水线图形表示
单周期流水线图
多周期流水线图
流水线控制单元
控制信号（同单周期）
跨流水级控制传送
带控制的单周期流水线图
流水线冒险
结构冒险
大型分析题
数据冒险
数据相关（数据依赖）
寄存器先写后读
旁路
第1类：ALU-ALU旁路
第2类：MEM-ALU旁路
附加条件
阻塞
取数-使用型数据冒险
nop气泡指令
控制冒险
假定分支发生/不发生
动态分支预测
分支预测缓存（分支历史记录）
单/双预测位
缩短分支延迟（ID级分支）
分支目标地址计算
判断分支条件：额外的旁路与阻塞
带冒险控制的单周期流水线图
异常 ARM和x86处理器
指令级并行 矩阵乘法